

AWS EKS Troubleshooting - Teacher Edition

AWS EKS Troubleshooting - Break It, Diagnose It, Fix It

Teacher Edition for Classroom Students

Classroom Lab Conventions

Section 1 - Pod & Application Failures

1. Pod Stuck in Pending

1. OVERVIEW

2. HOW MANY WAYS THIS CAN HAPPEN

3. ROOT CAUSE 1 - Insufficient CPU

ROOT CAUSE 2 - Insufficient Memory

ROOT CAUSE 3 - NodeSelector Mismatch

ROOT CAUSE 4 - Untolerated Taint

ROOT CAUSE 5 - PVC Cannot Bind

ROOT CAUSE 6 - Too Many Pods / Pod-IP Capacity

4. VIDEO STRUCTURE SUGGESTION

5. COMMON MISTAKES / GOTCHAS

6. COMBINE-OR-SPLIT CALL

2. CrashLoopBackOff

1. OVERVIEW

2. HOW MANY WAYS THIS CAN HAPPEN

ROOT CAUSE 1 - Application Exits

ROOT CAUSE 2 - OOMKilled

ROOT CAUSE 3 - Liveness Probe Killing the App

ROOT CAUSE 4 - Missing Environment Variable

ROOT CAUSE 5 - Dependency Unavailable

4. VIDEO STRUCTURE

5. COMMON MISTAKES

6. COMBINE-OR-SPLIT

3. ImagePullBackOff

1. OVERVIEW

2. ROOT CAUSES

ROOT CAUSE 1 - Wrong Tag

ROOT CAUSE 2 - Private Registry Missing Credentials

ROOT CAUSE 3 - ECR Permissions

ROOT CAUSE 4 - Network Path to Registry Missing

4. VIDEO STRUCTURE

5. GOTCHAS

6. COMBINE-OR-SPLIT

4. ErrImagePull

1. OVERVIEW

2. ROOT CAUSES

3. BREAK / DIAGNOSE / FIX

4. VIDEO STRUCTURE

5. GOTCHAS

6. COMBINE-OR-SPLIT

5. Container Exits Immediately

OVERVIEW

ROOT CAUSES

ROOT CAUSE - Normal Process Completion

ROOT CAUSE - Executable Missing

VIDEO

GOTCHAS

COMBINE-OR-SPLIT

6. Pod Stuck in ContainerCreating

OVERVIEW

ROOT CAUSES

ROOT CAUSE - Missing Secret

ROOT CAUSE - VPC CNF Cannot Assign IP

VIDEO

COMBINE-OR-SPLIT

7. Pod Stuck in Terminating

OVERVIEW

ROOT CAUSES

ROOT CAUSE - Hanging preStop

ROOT CAUSE - Finalizer
VIDEO
COMBINE-OR-SPLIT

8. Pod Repeatedly Restarting

OVERVIEW
ROOT CAUSES
DIAGNOSE
FIX
VIDEO
COMBINE-OR-SPLIT

9. OOMKilled

OVERVIEW
ROOT CAUSES
BREAK
DIAGNOSE
FIX
VERIFY
VIDEO
COMBINE-OR-SPLIT

10. Exit Code 137

OVERVIEW
ROOT CAUSES
BREAK - Non-OOM 137
DIAGNOSE
FIX
VIDEO
COMBINE-OR-SPLIT

11. Exit Code 1

OVERVIEW
BREAK
DIAGNOSE
FIX
COMBINE-OR-SPLIT

12. Init Container Failure

OVERVIEW

ROOT CAUSES
BREAK
DIAGNOSE
FIX
VIDEO
COMBINE-OR-SPLIT

13. Sidecar Container Failure

OVERVIEW
BREAK
DIAGNOSE
FIX
COMBINE-OR-SPLIT

14. Application Works Locally but Fails in EKS

OVERVIEW
ROOT CAUSES
ROOT CAUSE - App Listens Only on localhost
ROOT CAUSE - AWS Credentials Different
VIDEO
COMBINE-OR-SPLIT

15. Application Starts but Pod Never Becomes Ready

OVERVIEW
ROOT CAUSES
BREAK
DIAGNOSE
FIX

Section 2 - Scheduling Problems

16. 0/x nodes are available

OVERVIEW
ROOT CAUSES
BREAK
DIAGNOSE
FIX
VIDEO
COMBINE-OR-SPLIT

- 17. Insufficient CPU
 - EXTRA TEACHING DEPTH
 - VIDEO
 - COMBINE-OR-SPLIT
- 18. Insufficient Memory
 - BREAK
 - DIAGNOSE
 - TEACHER COMPARISON
 - FIX
 - COMBINE-OR-SPLIT
- 19. NodeSelector Mismatch
 - BREAK
 - DIAGNOSE
 - FIX
 - VIDEO
 - COMBINE-OR-SPLIT
- 20. Node Affinity Prevents Scheduling
 - OVERVIEW
 - BREAK
 - DIAGNOSE
 - FIX
 - GOTCHA
 - COMBINE-OR-SPLIT
- 21. Pod Anti-Affinity Problems
 - OVERVIEW
 - BREAK
 - DIAGNOSE
 - FIX
 - COMBINE-OR-SPLIT
- 22. Taints and Tolerations
 - TAINT EFFECTS
 - BREAK
 - DIAGNOSE
 - FIX
 - IMPORTANT CLASSROOM POINT

[COMBINE-OR-SPLIT](#)

23. Pod Topology Spread Constraints

[OVERVIEW](#)

[BREAK](#)

[DIAGNOSE](#)

[FIX](#)

[COMBINE-OR-SPLIT](#)

24. PVC Prevents Scheduling

[OVERVIEW](#)

[ROOT CAUSES](#)

[DIAGNOSE](#)

[FIX](#)

[COMBINE-OR-SPLIT](#)

25. Too Many Pods on Node

[OVERVIEW](#)

[DIAGNOSE](#)

[FIX](#)

[VIDEO](#)

[COMBINE-OR-SPLIT](#)

Section 3 - Probes

26. Liveness Probe Failing

[OVERVIEW](#)

[ROOT CAUSES](#)

[BREAK](#)

[EXPLANATION](#)

[DIAGNOSE](#)

[FIX](#)

[COMBINE-OR-SPLIT](#)

27. Readiness Probe Failing

[BREAK](#)

[DIAGNOSE](#)

[FIX](#)

[COMBINE-OR-SPLIT](#)

28. Startup Probe Failing

[OVERVIEW](#)

- BREAK
- DIAGNOSE
- FIX
- COMBINE-OR-SPLIT

29. HTTP Probe Returning 404/500

- OVERVIEW
- BREAK
- DIAGNOSE
- FIX
- COMBINE-OR-SPLIT

30. Probe Timeout

- OVERVIEW
- ROOT CAUSES
- DIAGNOSE
- FIX
- COMBINE-OR-SPLIT

31. Incorrect Probe Port

- BREAK
- DIAGNOSE
- FIX

32. Application Startup Too Slow

- BREAK
- WHY IT BREAKS
- DIAGNOSE
- FIX

33. Readiness Probe 504

- OVERVIEW
- DIAGNOSE
- FIX
- COMBINE-OR-SPLIT

34. Liveness Probe Causing Restart Loops

- DIAGNOSE SEQUENCE
- FIX
- COMBINE-OR-SPLIT

Section 4 - Networking

35. Pod Cannot Communicate With Another Pod

OVERVIEW

BREAK LAB

ROOT CAUSES

DIAGNOSE

FIX

COMBINE-OR-SPLIT

36. Cross-Namespace Communication Fails

OVERVIEW

BREAK

DIAGNOSE

FIX

37. Service Not Reachable

OVERVIEW

DIAGNOSE

COMBINE-OR-SPLIT

38. ClusterIP Not Working

BREAK

DIAGNOSE

FIX

COMBINE-OR-SPLIT

39. DNS Resolution Failure

OVERVIEW

BREAK/TEST

DIAGNOSE

FIX

COMBINE-OR-SPLIT

40. CoreDNS Failure

ROOT CAUSES

DIAGNOSE

FIX

41. `nslookup` Works but Application Doesn't

OVERVIEW

DIAGNOSE

COMBINE-OR-SPLIT

42. NetworkPolicy Blocking Traffic

BREAK

DIAGNOSE

FIX

43. Ingress Returns 404

OVERVIEW

ROOT CAUSES

DIAGNOSE

COMBINE-OR-SPLIT

44. Ingress Returns 502

OVERVIEW

ROOT CAUSES

DIAGNOSE

FIX

COMBINE-OR-SPLIT

45. Ingress Returns 503

OVERVIEW

ROOT CAUSES

DIAGNOSE

FIX

46. LoadBalancer Stuck Pending

OVERVIEW

ROOT CAUSES

DIAGNOSE

IAM TEACHING

COMBINE-OR-SPLIT

47. Wrong Service Selector

BREAK

DIAGNOSE

FIX

COMBINE-OR-SPLIT

48. Service Has No Endpoints

ROOT CAUSES

DIAGNOSE

TEACHING POINT

49. TargetPort / Port Mismatch

[BREAK](#)

[DIAGNOSE](#)

[FIX](#)

50. External Service Inaccessible

[OVERVIEW](#)

[ROOT CAUSES](#)

[DIAGNOSE LADDER](#)

[COMBINE-OR-SPLIT](#)

51. kube-proxy Issue

[OVERVIEW](#)

[BREAK SAFELY](#)

[DIAGNOSE](#)

[FIX](#)

52. CNI Plugin Issue

[OVERVIEW](#)

[ROOT CAUSES](#)

[DIAGNOSE](#)

[FIX](#)

[COMBINE-OR-SPLIT](#)

Section 5 - Storage

53. PVC Stuck Pending

[OVERVIEW](#)

[ROOT CAUSES](#)

[BREAK - Missing StorageClass](#)

[DIAGNOSE](#)

[BREAK - EBS CSI IAM Failure](#)

[FIX](#)

54. PV Binding Failure

[ROOT CAUSES](#)

[BREAK](#)

[DIAGNOSE](#)

[FIX](#)

55. StorageClass Missing

[BREAK](#)

DIAGNOSE

FIX

56. Wrong Access Mode

OVERVIEW

BREAK

DIAGNOSE

FIX

57. Volume Mount Failure

OVERVIEW

ROOT CAUSES

DIAGNOSE

FIX

58. FailedMount

OVERVIEW

BREAK

DIAGNOSE

FIX

COMBINE-OR-SPLIT

59. FailedAttachVolume

OVERVIEW

ROOT CAUSES

DIAGNOSE

FIX

60. Multi-Attach Error

BREAK

DIAGNOSE

FIX

61. StatefulSet Volume Problems

OVERVIEW

ROOT CAUSES

DIAGNOSE

FIX

62. Pod Moved to Another Node and Volume Won't Attach

OVERVIEW

DIAGNOSE

[FIX](#)

63. Disk Full

[OVERVIEW](#)

[DIAGNOSE INSIDE POD](#)

[FIX](#)

64. Permission Denied on Mounted Volume

[BREAK](#)

[DIAGNOSE](#)

[FIX](#)

Section 6 - Resources & Scaling

65. CPU Throttling

[OVERVIEW](#)

[BREAK](#)

[DIAGNOSE](#)

[FIX](#)

66. Memory Limit Exceeded

[TEACHING DIFFERENCE](#)

[DIAGNOSE](#)

[FIX](#)

[COMBINE](#)

67. Requests Too High

[BREAK](#)

[DIAGNOSE](#)

[FIX](#)

68. Limits Incorrectly Configured

[ROOT CAUSES](#)

[DIAGNOSE](#)

[COMBINE-OR-SPLIT](#)

69. HPA Not Scaling

[OVERVIEW](#)

[ROOT CAUSES](#)

[DIAGNOSE](#)

[FIX](#)

70. Metrics Server Unavailable

OVERVIEW

DIAGNOSE

FIX

71. HPA Shows <unknown>

OVERVIEW

DIAGNOSE

FIX

72. Cluster Autoscaler Doesn't Add Nodes

OVERVIEW

ROOT CAUSES

BREAK

DIAGNOSE

FIX

COMBINE-OR-SPLIT

73. Pod Can't Schedule After HPA Scaling

OVERVIEW

BREAK

DIAGNOSE

FIX

Section 7 - Configuration

74. ConfigMap Not Loaded

ROOT CAUSES

BREAK

DIAGNOSE

FIX

75. Secret Not Available

ROOT CAUSES

DIAGNOSE

FIX

76. Environment Variable Missing

BREAK

DIAGNOSE

FIX

77. Incorrect YAML Indentation

OVERVIEW

DIAGNOSE BEFORE APPLY

FIX

78. Invalid API Version

BREAK

DIAGNOSE

FIX

COMBINE-OR-SPLIT

79. Strict Decoding Error

BREAK

DIAGNOSE

FIX

80. Typo Such as `limmits`

CLASSROOM EXERCISE

COMBINE-OR-SPLIT

81. Deployment Selector Mismatch

BREAK

DIAGNOSE

FIX

82. Wrong Namespace

BREAK

DIAGNOSE

FIX

Section 8 - RBAC and Security

83. Forbidden Error

OVERVIEW

DIAGNOSE

FIX

COMBINE-OR-SPLIT

84. ServiceAccount Permission Denied

OVERVIEW

BREAK - Kubernetes RBAC

FIX RBAC

AWS IAM CASE

TEACHING POINT

85. Role vs ClusterRole

OVERVIEW

BREAK

DIAGNOSE

FIX

86. RoleBinding Misconfiguration

ROOT CAUSES

DIAGNOSE

FIX

87. Image Registry Authentication

DIAGNOSE

COMBINE

88. ImagePullSecret Problems

BREAK

DIAGNOSE

FIX

89. Certificate Expired

OVERVIEW

DIAGNOSE EXTERNAL TLS

FIX

90. TLS Certificate Error

ROOT CAUSES

DIAGNOSE

FIX

Section 9 - Node and Control Plane

91. Node NotReady

OVERVIEW

ROOT CAUSES

BREAK LAB

DIAGNOSE

FIX

COMBINE-OR-SPLIT

92. kubelet Not Running

ADDITIONAL DIAGNOSIS

93. kubelet node not found

OVERVIEW

BREAK LAB

DIAGNOSE

FIX

94. DiskPressure

OVERVIEW

SAFE BREAK LAB

DIAGNOSE

FIX

95. MemoryPressure

OVERVIEW

DIAGNOSE

FIX

96. PIDPressure

OVERVIEW

SAFE BREAK LAB

DIAGNOSE

97. API Server Unavailable

OVERVIEW

DIAGNOSE

FIX

98. kubectl Connection Refused

OVERVIEW

DIAGNOSE

FIX

99. localhost:8080 refused

OVERVIEW

BREAK LAB

DIAGNOSE

FIX

- 100. etcd Unavailable - EKS Adaptation
 - OVERVIEW
 - WHAT SHOULD STUDENTS TROUBLESHOOT INSTEAD?
 - DIAGNOSE
 - FIX
 - VIDEO
- 101. etcd TLS Handshake Failure - EKS Adaptation
 - OVERVIEW
 - DIAGNOSE CLIENT TRUST
 - COMBINE-OR-SPLIT
- 102. etcd Port 2379 Already in Use - EKS Adaptation
 - OVERVIEW
 - CLASSROOM LESSON
 - COMBINE-OR-SPLIT
- 103. Port 10250 Already in Use
 - OVERVIEW
 - BREAK LAB - DISPOSABLE WORKER ONLY
 - DIAGNOSE
 - FIX
- 104. CoreDNS Not Ready
 - OVERVIEW
 - DIAGNOSE
 - FIX
- 105. kube-proxy Failure
 - ADDITIONAL TEACHING
- 106. CNI Not Initialized
 - OVERVIEW
 - DIAGNOSE
 - FIX
- 107. Certificate Problems
 - OVERVIEW
 - DIAGNOSE KUBELET
 - DIAGNOSE kubectl
 - FIX

108. kubeadm Upgrade Problems - EKS Managed Upgrade Version

OVERVIEW

ROOT CAUSE 1 - Deprecated API

ROOT CAUSE 2 - EKS Add-on Compatibility

ROOT CAUSE 3 - Managed Node Group Behind Control Plane

ROOT CAUSE 4 - PDB Blocks Drain

ROOT CAUSE 5 - New Node Doesn't Join

ROOT CAUSE 6 - Insufficient Temporary Capacity

VIDEO STRUCTURE

COMMON MISTAKES

COMBINE-OR-SPLIT

Recommended Classroom Troubleshooting Pattern for Every Demo

Step 1 - Identify the object

Step 2 - Get detailed state

Step 3 - Check Events

Step 4 - Check application/container evidence

Step 5 - Check the live runtime

Step 6 - Check cluster component

Step 7 - Check AWS

Step 8 - Change One Thing

Step 9 - Verify the Fix

How I Would Teach These Topics to Students

Phase 1 - Healthy Environment

Phase 2 - Explain the Architecture

Phase 3 - Break Exactly One Layer

Phase 4 - Observe Without Fixing

Phase 5 - Collect Evidence

Phase 6 - Form the Root-Cause Statement

Phase 7 - Fix

Phase 8 - Verify at Multiple Layers

Recommended Video Grouping

Current AWS EKS References Used in the Material

AWS EKS Troubleshooting - Break It, Diagnose It, Fix It

Teacher Edition for Classroom Students

This handbook is designed for classroom teaching. Every troubleshooting topic follows the same reasoning pattern: first understand the symptom, then deliberately create the failure, explain exactly why the break step causes the issue, diagnose the failure with evidence, explain why each command is being run and what it proves, apply the fix with an explanation of why it works, and finally verify that the workload or cluster component has recovered.

Classroom Lab Conventions

Use a disposable EKS cluster for the exercises.

```
export CLUSTER=breakfix-eks
export REGION=ap-south-1
export NS=breakfix

kubectl create namespace "$NS" \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl config set-context --current --namespace="$NS"
```

Useful variables:

```
export ACCOUNT_ID=$(aws sts get-caller-identity \
  --query Account \
  --output text)

export K8S_VERSION=$(aws eks describe-cluster \
  --name "$CLUSTER" \
  --region "$REGION" \
  --query cluster.version \
  --output text)

export NG=$(aws eks list-nodegroups \
  --cluster-name "$CLUSTER" \
  --region "$REGION" \
  --query 'nodegroups[0]' \
  --output text)
```

Before every troubleshooting lab, teach students this sequence:

```
SYMPTOM
↓
Kubernetes object status
↓
Events
↓
Container logs
↓
Pod configuration
↓
Node / networking / storage layer
↓
AWS infrastructure
↓
Root cause
↓
Fix
↓
Verification
```

A good troubleshooter should never jump directly from:

Pod is Pending

to:

Let's restart the pod.

The objective is to collect evidence first.

Section 1 - Pod & Application Failures

1. Pod Stuck in Pending

1. OVERVIEW

Pending means Kubernetes has accepted the Pod object but the pod has not yet reached the point where its containers can run.

Students should understand that **Pending is a phase, not a root cause**.

A pod can remain Pending because:

- the Kubernetes scheduler cannot find a suitable node;
- a required PVC cannot be satisfied;
- the pod requires a node label that doesn't exist;
- all nodes have incompatible taints;
- the cluster has reached pod/IP capacity;
- Fargate scheduling requirements are not met;
- additional node capacity isn't being created.

The first question should therefore be:

Has the pod been assigned to a node?

Check:

```
kubectl get pod <pod-name> -o wide
```

If `NODE` is empty, focus on scheduling.

If a node has already been assigned but the pod remains Pending/ContainerCreating, investigate kubelet, storage and CNI instead.

2. HOW MANY WAYS THIS CAN HAPPEN

Important causes for EKS:

1. Insufficient CPU
2. Insufficient memory
3. NodeSelector mismatch
4. Required node affinity cannot be satisfied
5. Untolerated taint
6. PVC cannot bind
7. Too many pods on the node
8. Fargate profile mismatch
9. Cluster Autoscaler cannot expand capacity
10. Topology/AZ restriction

3. ROOT CAUSE 1 - Insufficient CPU

A. BREAK IT

```
kubectl run pending-cpu \
  --image=nginx \
  --requests='cpu=100000m,memory=64Mi'
```

What does this command do?

The command creates a pod using nginx.

The important part is:

```
cpu=100000m
```

`1000m` equals one CPU core.

Therefore:

```
100000m = 100 CPU cores
```

Unless the cluster has a worker node with 100 allocatable CPUs, no node can satisfy the request.

Why will this break scheduling?

The Kubernetes scheduler does not initially ask:

```
How much CPU is the node currently using?
```

Instead, it checks whether there is enough **unallocated requested CPU** remaining.

Our pod is asking the scheduler to reserve 100 cores.

The scheduler evaluates every node and rejects each one.

B. DIAGNOSE IT

Start with:

```
kubectl get pod pending-cpu
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
pending-cpu	0/1	Pending	0	20s

This confirms only the symptom.

Now:

```
kubectl describe pod pending-cpu
```

Why do we run describe?

`kubectl get` tells us **what state the pod is in**.

`kubectl describe` gives us:

- requested resources;
- scheduling configuration;
- node assignment;
- conditions;
- recent Events.

Look near the bottom:

```
Events:
  Warning  FailedScheduling
    0/3 nodes are available: 3 Insufficient cpu
```

This is the most important evidence.

The scheduler is explicitly saying:

```
Insufficient cpu
```

Now inspect node capacity:

```
kubectl get nodes
```

Then:

```
kubectl describe node <node-name>
```

Look for:

```
Capacity:
  cpu:

Allocatable:
  cpu:
```

Capacity vs Allocatable

Teach students:

```
Capacity
  = total hardware resource

Allocatable
  = resource Kubernetes can give pods
```

Kubernetes reserves some resources for the OS and system processes.

Also inspect requested CPU:

```
kubectl describe node <node-name>
```

Look for:

```
Allocated resources:
  Resource  Requests
  cpu
```

Finally:

```
kubectl top nodes
```

Why run `kubectl top`?

It shows **actual usage**.

This is useful for demonstrating an important lesson:

Scheduler decision -> based mainly on requests

`kubectl top` -> current real utilization

A node could be using only 25% CPU and still reject a pod because its requested CPU is already committed.

C. FIX IT

Delete the deliberately broken pod:

```
kubectl delete pod pending-cpu
```

Create one with a realistic request:

```
kubectl run pending-cpu \
  --image=nginx \
  --requests='cpu=100m,memory=64Mi'
```

Why does this fix it?

Now the scheduler only needs:

100m = 0.1 CPU

A normal EKS worker will likely have enough unallocated requested CPU.

Verify:

```
kubectl get pod pending-cpu -o wide
```

Expected:

Running

Also verify the previous error has disappeared:

```
kubectl describe pod pending-cpu
```

Alternative production fixes

If the request is genuinely required, don't reduce it artificially.

Options include:

- larger EC2 instance type;
- more managed-node-group nodes;
- Cluster Autoscaler;
- Karpenter;
- EKS Auto Mode compute;
- moving large workloads to dedicated node groups.

ROOT CAUSE 2 - Insufficient Memory

A. BREAK IT

```
kubectl run pending-memory \
  --image=nginx \
  --requests='cpu=50m,memory=1000Gi'
```

What are we doing?

The pod asks Kubernetes to reserve:

```
1000 GiB RAM
```

No normal worker can satisfy it.

Why does it break?

Scheduling considers **memory requests**.

The scheduler discovers that every worker has less free allocatable requested memory than the pod requires.

B. DIAGNOSE

```
kubectl describe pod pending-memory
```

Look for:

```
0/3 nodes are available:
3 Insufficient memory
```

Inspect node memory:

```
kubectl describe nodes
```

Look for:

```
Capacity:
Allocatable:
Allocated resources:
```

Current usage:

```
kubectl top nodes
```

Teaching point

Make students distinguish:

```
Insufficient memory
```

from:

```
OOMKilled
```

Insufficient memory occurs **before scheduling**.

OOMKilled occurs **after a container has started**.

C. FIX

```
kubectl delete pod pending-memory
```

Recreate:

```
kubectl run pending-memory \
  --image=nginx \
  --requests='cpu=50m,memory=64Mi'
```

Verify:

```
kubectl get pod pending-memory
```

ROOT CAUSE 3 - NodeSelector Mismatch

A. BREAK IT

```
kubectl run selector-demo \
  --image=nginx \
  --overrides='{
    "spec":{
      "nodeSelector":{
        "workload":"gpu"
      }
    }
  }'
```

What did we change?

We told the Kubernetes scheduler:

```
This pod is allowed to run only on nodes containing the label  
workload=gpu.
```

Now check:

```
kubectl get nodes -L workload
```

If no node contains:

```
workload=gpu
```

the pod has nowhere to run.

B. DIAGNOSE

```
kubectl get pod selector-demo
```

Then:

```
kubectl describe pod selector-demo
```

Expected scheduler evidence:

```
node(s) didn't match Pod's node affinity/selector
```

Now:

```
kubectl get nodes --show-labels
```

Why?

We're testing the scheduler's statement.

The pod requires:

```
workload=gpu
```

We need to prove whether any node actually possesses that label.

C. FIX

For the lab:

```
NODE=$(kubectl get nodes \
-o jsonpath='{.items[0].metadata.name}')

kubectl label node "$NODE" workload=gpu
```

What does this do?

It adds:

```
workload=gpu
```

to one worker.

The scheduler now has a valid destination.

Verify:

```
kubectl get pod selector-demo -w
```

It should transition:

```
Pending
-> ContainerCreating
-> Running
```

Production teaching point:

For managed node groups, persist workload labels through node-group/laC configuration rather than manually labeling individual EC2 nodes that EKS may later replace.

ROOT CAUSE 4 - Untolerated Taint

A. BREAK IT

Choose a disposable node:

```
NODE=$(kubectl get nodes \
-o jsonpath='{.items[0].metadata.name}')
```

Apply:

```
kubectl taint node "$NODE" \
dedicated=database:NoSchedule
```

What does this mean?

The node now says:

Do not schedule new pods here unless they explicitly tolerate `dedicated=database`.

A taint belongs to the **node**.

A toleration belongs to the **pod**.

B. DIAGNOSE

```
kubectl describe node "$NODE"
```

Look for:

```
Taints:
dedicated=database:NoSchedule
```

Then:

```
kubectl describe pod <pending-pod>
```

Possible scheduler output:

```
node(s) had untolerated taint
```

Why these commands?

First we prove the node actually has the taint.

Then we check whether the scheduler is specifically rejecting the pod because it lacks the matching toleration.

C. FIX

Add:

```
tolerations:
- key: dedicated
  operator: Equal
  value: database
  effect: NoSchedule
```

Why does this work?

The pod tells Kubernetes:

I understand and accept this particular taint.

It becomes eligible for the node.

Important:

```
Toleration != placement
```

A toleration **allows** scheduling.

It does not force scheduling.

Alternatively remove the lab taint:

```
kubectl taint node "$NODE" \
  dedicated=database:NoSchedule-
```

ROOT CAUSE 5 - PVC Cannot Bind

A. BREAK IT

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: broken-pvc
  namespace: breakfix
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: class-does-not-exist
  resources:
    requests:
      storage: 1Gi
```

Apply:

```
kubectl apply -f broken-pvc.yaml
```

Why will it break?

The PVC explicitly asks for:

```
class-does-not-exist
```

If the StorageClass does not exist, Kubernetes cannot dynamically provision the volume.

B. DIAGNOSE

```
kubectl get pvc
```

Expected:

```
Pending
```

Now:

```
kubectl describe pvc broken-pvc
```

Why?

PVC Events normally explain why the storage provisioner cannot satisfy the claim.

Check available classes:

```
kubectl get storageclass
```

Now compare:

PVC asks for
class-does-not-exist

Cluster contains
gp3

That proves the mismatch.

C. FIX

Delete:

```
kubectl delete pvc broken-pvc
```

Determine valid EBS StorageClass:

```
kubectl get sc
```

Recreate using the valid class.

For example:

```
storageClassName: gp3
```

Verify:

```
kubectl get pvc -w
```

Expected transition:

Pending -> Bound

ROOT CAUSE 6 - Too Many Pods / Pod-IP Capacity

A. BREAK IT

First check each node's pod capacity:

```
kubectl get nodes \
-o custom-columns=NAME:.metadata.name,POD-CAPACITY:.status.allocatable.pods
```

On traditional EC2-backed EKS using the Amazon VPC CNI, pod networking is closely related to the ENI/IP capacity of the instance. The VPC CNI runs on EC2 nodes, manages ENIs and assigns private VPC IP addresses to pods.

To demonstrate safely, use a small disposable node group and schedule many tiny pods.

B. DIAGNOSE

```
kubectl describe pod <pending-pod>
```

Look for:

```
Too many pods
```

Count pods on the node:

```
kubectl get pods -A \
--field-selector spec.nodeName="$NODE" \
--no-headers | wc -l
```

Check CNI:

```
kubectl -n kube-system get ds aws-node
```

Check IP availability:

```
SUBNETS=$(aws eks describe-cluster \
--name "$CLUSTER" \
--region "$REGION" \
--query 'cluster.resourcesVpcConfig.subnetIds' \
--output text)

aws ec2 describe-subnets \
--subnet-ids $SUBNETS \
--region "$REGION" \
--query 'Subnets[].{Subnet:SubnetId,FreeIPs:AvailableIpAddressCount}'
```

Why check AWS subnets?

Even if Kubernetes nodes have CPU and memory available, the CNI may have no usable VPC addresses for additional pods.

C. FIX

Possible fixes:

- scale out worker nodes;
- select instances with greater networking capacity;
- use VPC CNI prefix delegation where appropriate;
- add subnet address capacity;
- redesign pod density;
- evaluate newer EKS compute models where applicable.

Verify:

```
kubectl get pods -A -o wide
```

4. VIDEO STRUCTURE SUGGESTION

Suggested teaching order:

1. explain what Pending actually means;
2. CPU and memory;
3. selector/affinity;
4. taints;
5. PVC;
6. EKS networking/pod capacity;
7. autoscaling/Fargate.

Hook:

“Pending isn't the error. Pending only tells us Kubernetes hasn't started the workload yet. The Events tell us why.”

Suggested title:

EKS Pod Stuck in Pending? Break and Fix Every Major Cause

Suggested chapters:

00:00 What Pending Really Means
03:30 CPU and Memory Scheduling
10:00 Node Selectors and Affinity
16:00 Taints and Tolerations
22:00 PVC Scheduling
28:00 EKS ENI and Pod Limits
34:00 Autoscaling Problems

5. COMMON MISTAKES / GOTCHAS

Students commonly:

- run `kubectl logs` even though no container has started;
- restart the pod repeatedly;
- assume low `kubectl top CPU` means scheduling should succeed;
- immediately add nodes;
- ignore Events;
- confuse PVC Pending with pod CPU scheduling;
- confuse pod capacity with CPU capacity.

Always teach:

```
kubectl describe pod <pod>
```

before making changes.

6. COMBINE-OR-SPLIT CALL

Standalone flagship video.

Use the specialized scheduling topics below as deeper follow-up lessons.

2. CrashLoopBackOff

1. OVERVIEW

`CrashLoopBackOff` means Kubernetes successfully started the container, but the process repeatedly terminates and Kubernetes is delaying subsequent restart attempts.

Important student concept:

```
CrashLoopBackOff is not the root cause.
```

The real cause might be:

- application exit;
- missing configuration;

- OOM;
- liveness probe;
- failed dependency;
- permissions;
- IAM failure.

2. HOW MANY WAYS THIS CAN HAPPEN

1. Application exits with code 1
2. OOMKilled
3. Liveness probe kills container
4. Missing environment/configuration
5. Filesystem permission failure
6. AWS API permission problem
7. Database/dependency unavailable
8. Incorrect startup command

ROOT CAUSE 1 - Application Exits

A. BREAK IT

```
kubectl run crash-demo \
  --image=busybox:1.36 \
  --restart=Always \
  -- sh -c 'echo "Application configuration invalid"; exit 1'
```

What did we deliberately do?

BusyBox starts `/bin/sh`.

The shell:

1. prints an error;
2. exits with status 1.

Because restart policy is `Always`, kubelet starts it again.

It fails again.

Repeated failures eventually cause Kubernetes to apply increasing restart delays.

That's the “backoff.”

B. DIAGNOSE

```
kubectl get pod crash-demo
```

You may eventually see:

```
CrashLoopBackOff
```

Now:

```
kubectl describe pod crash-demo
```

Look for:

```
Last State:
  Terminated:
    Reason: Error
    Exit Code: 1
```

Then:

```
kubectl logs crash-demo
```

And most importantly:

```
kubectl logs crash-demo --previous
```

Why `--previous`?

When the container restarts, its current log may belong to the newly started instance.

`--previous` requests logs from the **previous terminated container instance**.

Expected:

```
Application configuration invalid
```

Students now have three pieces of evidence:

```
CrashLoopBackOff
+
Exit Code 1
+
Application error message
```

C. FIX

Delete the broken workload:

```
kubectl delete pod crash-demo
```

Create a process that remains alive:

```
kubectl run crash-demo \
  --image=busybox:1.36 \
  --restart=Always \
  -- sh -c 'while true; do sleep 3600; done'
```

Why does it work?

PID 1 remains alive rather than terminating.

Verify:

```
kubectl get pod crash-demo
```

Expected:

Running

Check:

```
kubectl get pod crash-demo \
  -o jsonpath='{.status.containerStatuses[0].restartCount}{"\n"}'
```

ROOT CAUSE 2 - OOMKilled

A. BREAK IT

```
apiVersion: v1
kind: Pod
metadata:
  name: crash-oom
  namespace: breakfix
spec:
  restartPolicy: Always
  containers:
  - name: app
    image: python:3.12-alpine
    command:
    - python
    - -c
    - |
      import time
      x = bytearray(200 * 1024 * 1024)
      time.sleep(300)
  resources:
    limits:
      memory: 64Mi
```

What does the application do?

It attempts to allocate roughly:

200 MiB

but Kubernetes gives the container a memory limit of:

64 MiB

The kernel eventually kills the process when the cgroup exceeds the permitted memory.

B. DIAGNOSE

```
kubectl describe pod crash-oom
```

Look for:

```
Reason: OOMKilled
Exit Code: 137
```

Or:

```
kubectl get pod crash-oom \
-o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}{" "}{.status.containerStatuses[0].lastState.terminated.exitCode}{"\n"}'
```

Expected:

```
OOMKilled 137
```

Why is Reason more important than exit code alone?

Exit 137 means the process received SIGKILL.

OOM is one reason for SIGKILL, but it is not the only reason.

OOMKilled provides stronger evidence.

C. FIX

Raise the limit according to measured workload requirements:

```
resources:
  requests:
    memory: 128Mi
  limits:
    memory: 256Mi
```

Then:

```
kubectl apply -f app.yaml
```

Verify:

```
kubectl get pods -w
kubectl top pod crash-oom --containers
```

ROOT CAUSE 3 - Liveness Probe Killing the App

A. BREAK IT

Create nginx with:

```
livenessProbe:
  httpGet:
    path: /this-does-not-exist
    port: 80
  initialDelaySeconds: 2
  periodSeconds: 3
  failureThreshold: 1
```

What are we changing?

Nginx is healthy.

But Kubernetes checks:

```
/this-does-not-exist
```

Nginx returns:

```
404
```

Because this is a **liveness** check, kubelet interprets repeated failure as:

```
The application is unhealthy and should be restarted.
```

So Kubernetes creates the outage itself.

B. DIAGNOSE

```
kubectl describe pod <pod>
```

Look for:

```
Liveness probe failed:
HTTP probe failed with statuscode: 404
```

and:

```
Killing container
```

Then:

```
kubectl get pod <pod> -w
```

Watch the restart counter increase.

Manually test:

```
kubectl exec <pod> -- \
  wget -S -O- http://127.0.0.1/this-does-not-exist
```

Why manually run the probe?

It proves that Kubernetes is correctly reporting the application's HTTP response.

C. FIX

Change:

```
path: /
```

or your real liveness endpoint.

Apply:

```
kubectl apply -f deployment.yaml
```

Verify:

```
kubectl describe pod <new-pod>
```

There should be no new liveness failures.

ROOT CAUSE 4 - Missing Environment Variable

A. BREAK IT

Create:

```
command:
- sh
- -c
- |
  if [ -z "$DATABASE_URL" ]; then
    echo "DATABASE_URL is missing"
    exit 1
  fi
  sleep 3600
```

Do not define `DATABASE_URL`.

Why does this break?

The container technically starts correctly.

The **application's startup validation** detects an incomplete environment and terminates itself.

B. DIAGNOSE

```
kubectl logs <pod> --previous
```

Expected:

```
DATABASE_URL is missing
```

Inspect Deployment:

```
kubectl get deployment <deployment> -o yaml
```

Look under:

```
spec.template.spec.containers[].env
```

C. FIX

Add:

```
env:  
- name: DATABASE_URL  
  value: "postgresql://db.example.internal:5432/app"
```

Better: reference a Secret where credentials are involved.

Verify after rollout:

```
kubectl exec <pod> -- printenv DATABASE_URL
```

Never display real production passwords in classroom demos.

ROOT CAUSE 5 - Dependency Unavailable

A. BREAK IT

Configure:

```
DATABASE_HOST=database.invalid
```

Have the application exit when the DB connection fails.

Why?

Applications frequently work as containers but fail during Kubernetes startup because they depend on external systems such as:

- RDS;
- Redis;
- Kafka;
- third-party APIs.

B. DIAGNOSE

Read application logs:

```
kubectl logs <pod> --previous
```

Then separate DNS from TCP:

```
kubectl exec <debug-pod> -- \
nslookup database.invalid
```

and:

```
kubectl exec <debug-pod> -- \
nc -vZ database.example.internal 5432
```

Why two commands?

nslookup answers:

Can the hostname resolve?

nc answers:

Can we establish a TCP connection to that port?

They test different layers.

C. FIX

Correct the hostname, networking or security groups.

Also teach application resilience:

A temporary RDS outage should usually cause retries/backoff rather than immediate permanent application death.

4. VIDEO STRUCTURE

Title:

EKS CrashLoopBackOff - Reproduce 8 Real Causes and Fix Them

Hook:

“CrashLoopBackOff is Kubernetes telling you that something keeps dying. Your job is to discover who killed it and why.”

Chapters:

```
What CrashLoopBackOff Means
Exit Code Troubleshooting
OOMKilled
Liveness Probe Failures
Configuration Problems
AWS and Database Dependencies
```

5. COMMON MISTAKES

Students often:

- delete pods repeatedly;
- inspect only current logs;
- assume Kubernetes itself crashed;
- treat exit code 137 as always OOM;
- overlook liveness;
- ignore sidecars/init containers.

Always teach:

```
kubectl logs <pod> --previous
kubectl describe pod <pod>
```

6. COMBINE-OR-SPLIT

Standalone flagship video.

OOMKilled, probes and exit codes deserve dedicated follow-ups.

3. ImagePullBackOff

1. OVERVIEW

Before a container can start, the node needs its image.

`ImagePullBackOff` means image retrieval has failed repeatedly and kubelet is temporarily delaying another attempt.

No application logs normally exist because the application has not started.

2. ROOT CAUSES

1. Image doesn't exist
2. Tag doesn't exist
3. Private registry authentication missing
4. ECR authorization missing
5. Cross-account ECR policy problem
6. Registry unavailable
7. Node has no registry network path
8. TLS/CA failure
9. Registry rate limiting

ROOT CAUSE 1 - Wrong Tag

BREAK IT

```
kubectll run image-demo \  
  --image=nginx:no-such-version-12345
```

What does this do?

The pod specification asks the container runtime to retrieve an image tag that does not exist.

Kubernetes accepts the pod because Kubernetes does not validate every external registry tag during object creation.

The error appears later on the worker while attempting the pull.

DIAGNOSE

```
kubectll get pod image-demo
```

You may first catch:

```
ErrImagePull
```

and later:

```
ImagePullBackOff
```

Now:

```
kubectl describe pod image-demo
```

Look for:

```
Failed to pull image  
manifest unknown  
not found
```

Why describe, not logs?

There is no running application container to produce logs.

Image-pull messages are kubelet/container-runtime Events.

FIX

```
kubectl set image pod/image-demo \  
image-demo=nginx:stable
```

Verify:

```
kubectl get pod image-demo -w
```

ROOT CAUSE 2 - Private Registry Missing Credentials

BREAK

Reference a private registry without:

```
imagePullSecrets:
```

DIAGNOSE

```
kubectl describe pod <pod>
```

Look for:

```
unauthorized  
authentication required  
pull access denied
```

This is different from:

```
manifest unknown
```

The first means:

Registry knows the request but refuses access.

The second usually means:

Repository/tag does not exist.

FIX

```
kubectl create secret docker-registry regcred \  
  --docker-server=registry.example.com \  
  --docker-username="$REGISTRY_USER" \  
  --docker-password="$REGISTRY_PASSWORD"
```

Then:

```
imagePullSecrets:  
- name: regcred
```

Explain that an ImagePullSecret is namespace-scoped.

Verify:

```
kubectl get secret regcred  
kubectl rollout restart deployment/<deployment>
```

ROOT CAUSE 3 - ECR Permissions

BREAK

Use a private ECR image while deliberately using a lab node role without the required ECR pull permission.

DIAGNOSE

First identify node:

```
kubectl get pod <pod> -o wide
```

Identify node-group role:

```
NODE_ROLE_ARN=$(aws eks describe-nodegroup \  
  --cluster-name "$CLUSTER" \  
  --nodegroup-name "$NG" \  
  --region "$REGION" \  
  --query nodegroup.nodeRole \  
  --output text)  
  
echo "$NODE_ROLE_ARN"
```

List policies:

```
NODE_ROLE=${NODE_ROLE_ARN##*/}

aws iam list-attached-role-policies \
  --role-name "$NODE_ROLE"
```

Why check node IAM?

The image is normally pulled by the worker/container runtime before the application pod starts.

Therefore, giving the application's AWS SDK role permission does not necessarily solve EC2 worker ECR image retrieval.

FIX

Use the appropriate ECR pull permissions for the node role.

Then delete/recreate the failed pod to make the demonstration obvious:

```
kubectl delete pod <pod>
```

Verify Events.

ROOT CAUSE 4 - Network Path to Registry Missing

BREAK

A common real EKS architecture is private workers without public IP addresses.

If they have:

- no NAT path,
- and no required VPC endpoints,

they may not reach external registries/AWS services.

DIAGNOSE

Determine node subnet:

```
kubectl get pod <pod> -o wide
kubectl get node <node> -o jsonpath='{.spec.providerID}{"\n"}'
```

Inspect routes:

```
aws ec2 describe-route-tables \
  --region "$REGION"
```

For ECR private access also inspect relevant endpoints:

```
aws ec2 describe-vpc-endpoints \
  --region "$REGION"
```

Teaching lesson

Authentication and networking can produce superficially similar image-pull failures.

Always read the complete Event text.

FIX

Provide the architecture's intended egress route:

- NAT;
- appropriate AWS VPC endpoints;
- registry mirror;
- approved corporate proxy.

4. VIDEO STRUCTURE

Title:

EKS ImagePullBackOff - Bad Tags, ECR IAM, Secrets and Network Failures

Hook:

"If the image never reached the node, debugging your application code is impossible."

5. GOTCHAS

Do not:

```
kubectl logs <pod>
```

and conclude logging is broken.

There is no application container yet.

6. COMBINE-OR-SPLIT

Combine ImagePullBackOff + ErrImagePull + registry authentication + ImagePullSecret into one complete image-pull troubleshooting video.

4. ErrImagePull

1. OVERVIEW

`ErrImagePull` represents an image pull attempt that has failed.

`ImagePullBackOff` usually appears after Kubernetes begins backing off between retries.

Teach this timeline:

```
Pull image
↓
Failure
↓
ErrImagePull
↓
Retry/backoff
↓
ImagePullBackOff
↓
Retry
```

2. ROOT CAUSES

Same core causes as `ImagePullBackOff`:

1. Wrong repository
2. Wrong tag
3. Authentication
4. Authorization
5. Registry network access
6. TLS trust
7. Registry throttling

3. BREAK / DIAGNOSE / FIX

BREAK

```
kubectl run errimage-demo \
  --image=nginx:definitely-invalid-tag
```

Immediately watch:

```
kubectl get pod errimage-demo -w
```

Why watch?

If you wait too long, you may see only `ImagePullBackOff`.

Watching live helps students see both states.

Possible sequence:

```
ContainerCreating
ErrImagePull
ImagePullBackOff
ErrImagePull
ImagePullBackOff
```

DIAGNOSE

```
kubectl describe pod errimage-demo
```

Read the Event message carefully.

For example:

```
manifest unknown
```

confirms image/tag.

Whereas:

```
unauthorized
```

points to credentials.

FIX

```
kubectl set image pod/errimage-demo \
  errimage-demo=nginx:stable
```

Verify:

```
kubectl get pod errimage-demo -w
```

4. VIDEO STRUCTURE

Title:

ErrImagePull vs ImagePullBackOff on EKS - The Exact Difference

5. GOTCHAS

They are normally not two independent problems.

They are commonly two stages of one failed image-pull cycle.

6. COMBINE-OR-SPLIT

Combine with ImagePullBackOff.

5. Container Exits Immediately

OVERVIEW

This is different from image-pull failure.

The container:

```
successfully starts  
-> PID 1 runs  
-> PID 1 terminates
```

Containers remain alive only while their main process remains alive.

ROOT CAUSES

1. Command intentionally finishes
2. Application exits code 1
3. Wrong command
4. Executable missing
5. Configuration validation fails
6. Permission failure
7. OOM/SIGKILL

ROOT CAUSE - Normal Process Completion

BREAK

```
kubectl run exits-zero \  
  --image=busybox:1.36 \  
  --restart=Never \  
  -- echo "Task finished"
```

Explanation

`echo` runs, prints text, then exits successfully.

The container has nothing else to run.

Therefore the pod becomes:

```
Completed
```

DIAGNOSE

```
kubectl get pod exits-zero
```

Then:

```
kubectl describe pod exits-zero
```

Look for:

```
Reason: Completed  
Exit Code: 0
```

FIX

If this was intended as a web application, run a foreground server process instead.

If it was intended as a one-time task, `Completed` is correct and may belong in a Job rather than a Deployment.

ROOT CAUSE - Executable Missing

BREAK

```
kubectl run missing-command \  
  --image=busybox:1.36 \  
  --restart=Never \  
  -- /does/not/exist
```

DIAGNOSE

```
kubectl describe pod missing-command
```

Look for an error similar to:

```
executable file not found
```

Why?

Kubelet/containerd successfully creates the container environment, but the runtime cannot execute the requested binary.

FIX

Inspect the image's available executable/entrypoint and correct:

```
command:
```

and:

```
args:
```

VIDEO

EKS Container Starts and Immediately Stops - Understand PID 1 and Exit Codes

GOTCHAS

Completed isn't always failure.

For a Job:

```
Exit 0
```

is success.

For a Deployment web server, the same behavior is usually a configuration mistake.

COMBINE-OR-SPLIT

Combine with **Exit Code 1**.

6. Pod Stuck in ContainerCreating

OVERVIEW

This status is extremely valuable diagnostically.

It normally means:

```
Scheduler selected a node  
BUT  
kubelet has not completed container setup
```

Therefore investigate:

- networking;
- volumes;
- Secrets/ConfigMaps;
- container sandbox;
- runtime.

ROOT CAUSES

1. Missing Secret volume
2. Missing ConfigMap
3. CNI cannot allocate IP
4. EBS mount/attach failure
5. CSI node plugin unavailable

6. Container runtime failure

ROOT CAUSE - Missing Secret

BREAK

Use:

```
volumes:  
- name: credentials  
  secret:  
    secretName: secret-does-not-exist
```

Why does it break?

The pod is already scheduled.

Before launching the container, kubelet must prepare its required volumes.

The Secret volume cannot be constructed because the referenced Secret doesn't exist.

DIAGNOSE

```
kubectl get pod <pod> -o wide
```

Notice that a node has already been assigned.

Then:

```
kubectl describe pod <pod>
```

Look for:

```
MountVolume.SetUp failed  
secret "secret-does-not-exist" not found
```

Teaching point

Compare:

```
Pending with NODE=<none>
```

versus:

```
ContainerCreating with NODE=ip-...
```

The troubleshooting layer has changed.

FIX

```
kubectl create secret generic secret-does-not-exist \
  --from-literal=username=demo \
  --from-literal=password=demo
```

Kubelet will retry volume setup automatically.

Watch:

```
kubectl get pod <pod> -w
```

ROOT CAUSE - VPC CNI Cannot Assign IP

DIAGNOSE

```
kubectl describe pod <pod>
```

Possible event:

```
FailedCreatePodSandBox
failed to assign an IP address to container
```

Then:

```
kubectl -n kube-system get pods \
  -l k8s-app=aws-node -o wide
```

Why?

`aws-node` is the VPC CNI component on EC2-backed EKS workers.

Logs:

```
kubectl -n kube-system logs \
  ds/aws-node \
  -c aws-node \
  --tail=200
```

Subnet capacity:

```
aws ec2 describe-subnets \
  --subnet-ids $SUBNETS \
  --query 'Subnets[].{Subnet:SubnetId,AvailableIPs:AvailableIpAddressCount}'
```

FIX

Depending on evidence:

- restore CNI;
- correct CNI IAM;

- add subnet IP space;
- use prefix delegation;
- add nodes;
- fix ENI/API errors.

VIDEO

EKS Pod Stuck in ContainerCreating - CNI, EBS and Volume Failures

COMBINE-OR-SPLIT

Standalone.

7. Pod Stuck in Terminating

OVERVIEW

Deletion has been requested, but Kubernetes hasn't completed termination.

ROOT CAUSES

1. Long `terminationGracePeriodSeconds`
2. Hanging `preStop`
3. Finalizer
4. Dead/unreachable node
5. CSI volume cleanup
6. Stateful application shutdown

ROOT CAUSE - Hanging `preStop`

BREAK

```
lifecycle:
  preStop:
    exec:
      command:
        - sh
        - -c
        - sleep 300
```

What happens?

When deletion begins, kubelet invokes the `preStop` hook.

Our hook deliberately waits five minutes.

The pod remains Terminating while graceful shutdown proceeds.

DIAGNOSE

```
kubectl get pod <pod>
```

Then:

```
kubectl get pod <pod> -o yaml
```

Find:

```
deletionTimestamp:  
terminationGracePeriodSeconds:
```

Inspect lifecycle:

```
kubectl get pod <pod> \\  
-o jsonpath='{.spec.containers[*].lifecycle}'
```

FIX

Use a bounded shutdown action rather than an unnecessarily long hook.

For emergency lab cleanup only:

```
kubectl delete pod <pod> \\  
--grace-period=0 \\  
--force
```

Teaching warning

Force deletion removes the Kubernetes API object quickly.

It does **not necessarily prove**:

- old process stopped safely;
- storage detached;
- application flushed data.

ROOT CAUSE - Finalizer

BREAK

```
kubectl patch pod <pod> \\  
--type=merge \\  
-p '{"metadata":{"finalizers":["class.example/breakfix"]}}'
```

Then:

```
kubectl delete pod <pod>
```

Explanation

Finalizers tell Kubernetes:

Do not completely remove this object until the controller responsible for this finalizer finishes cleanup.

Our fake finalizer has no controller.

Therefore nobody removes it.

DIAGNOSE

```
kubectl get pod <pod> \
-o jsonpath='{.metadata.finalizers}{"\n"}'
```

This directly proves the deletion blocker.

FIX

Lab only:

```
kubectl patch pod <pod> \
--type=merge \
-p '{"metadata":{"finalizers":null}}'
```

Teach students never to blindly remove storage/cloud-resource finalizers in production because the cleanup represented by the finalizer may be important.

VIDEO

EKS Pod Stuck Terminating - Finalizers, preStop, Dead Nodes and EBS Cleanup

COMBINE-OR-SPLIT

Standalone.

8. Pod Repeatedly Restarting

OVERVIEW

Teach students to separate:

```
Container restart
```

from:

```
Pod replacement
```

A Deployment replacing a pod produces a new pod UID.

A container restart occurs inside the same pod.

ROOT CAUSES

1. Application exits
2. OOM
3. Liveness probe
4. Sidecar crashes
5. node disruption
6. Spot interruption
7. rolling deployment

DIAGNOSE

Check restart counts:

```
kubectl get pods
```

Detailed:

```
kubectl get pod <pod> \
-o jsonpath='{range .status.containerStatuses[*]}{.name}{ " restartCount="}
{.restartCount}{"\n"}{end}'
```

Check pod UID:

```
kubectl get pod <pod> \
-o jsonpath='{.metadata.uid}{"\n"}'
```

Why UID?

If the name/UID changes, a controller may have replaced the entire pod.

If UID stays the same while `restartCount` increases, the container itself is restarting.

FIX

Use the cause-specific evidence:

```
kubectl logs <pod> --previous  
kubectl describe pod <pod>  
kubectl get events --sort-by='.lastTimestamp'
```

VIDEO

Why Does My EKS Pod Keep Restarting? Container Restart vs Pod Replacement

COMBINE-OR-SPLIT

Combine with CrashLoopBackOff but keep the pod-replacement distinction as a dedicated chapter.

9. OOMKilled

OVERVIEW

`OOMKilled` means the container exceeded its allowed memory and was terminated by the Linux/kernel memory-control mechanism.

ROOT CAUSES

1. memory limit too low
2. memory leak
3. startup spike
4. JVM heap/off-heap mismatch
5. caching growth
6. application workload spike

BREAK

Use the Python allocation lab:

```
resources:  
  limits:  
    memory: 64Mi
```

while allocating 200 MiB.

Explanation

The container cannot use unlimited node RAM.

Its memory is constrained by the container's cgroup.

Crossing the limit can trigger termination.

DIAGNOSE

```
kubectl describe pod crash-oom
```

Look for:

```
OOMKilled
```

Check:

```
kubectl get pod crash-oom \
-o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'
```

Usage:

```
kubectl top pod crash-oom --containers
```

Additional investigation

For applications such as Java:

```
-Xmx
metaspace
direct buffers
thread stacks
native libraries
```

all matter.

A 512Mi heap inside a 512Mi container can still OOM because total process memory is greater than heap alone.

FIX

Increase the limit only after understanding expected usage:

```
resources:
  requests:
    memory: 256Mi
  limits:
    memory: 512Mi
```

Also investigate leaks/profiling.

VERIFY

```
kubectl get pod
kubectl top pod <pod> --containers
kubectl describe pod <pod>
```

Confirm restart count stops increasing.

VIDEO

OOMKilled on EKS - Memory Limits, Exit 137 and Real Diagnosis

COMBINE-OR-SPLIT

Standalone.

10. Exit Code 137

OVERVIEW

Linux exit codes above 128 commonly represent termination by a signal.

```
137 = 128 + 9
```

Signal 9 is:

```
SIGKILL
```

OOM is common, but 137 does **not automatically mean OOM**.

ROOT CAUSES

1. OOM kill
2. explicit SIGKILL
3. forced shutdown after grace timeout
4. node/container runtime termination
5. administrative kill

BREAK - Non-OOM 137

```
kubectl run exit137 \
  --image=busybox:1.36 \
  --restart=Never \
  -- sh -c 'kill -9 $$'
```

Explanation

The shell sends SIGKILL to itself.

No memory exhaustion occurs.

This gives students proof that:

```
Exit 137 != automatically OOM
```

DIAGNOSE

```
kubectl describe pod exit137
```

Then:

```
kubectl get pod exit137 \
-o jsonpath='{.status.containerStatuses[0].state.terminated.reason}' " "
{.status.containerStatuses[0].state.terminated.exitCode}{"\n"}
```

Compare this lab with an actual OOM pod.

FIX

Determine **why SIGKILL happened**.

For OOM:

- memory tuning.

For grace-period expiration:

- shutdown behavior.

For explicit kill:

- identify actor/process.

VIDEO

Exit Code 137 on EKS - Why It Isn't Always OOMKilled

COMBINE-OR-SPLIT

Combine with OOMKilled, but this makes an excellent Short.

11. Exit Code 1

OVERVIEW

Exit 1 means the application reported a general failure.

Kubernetes cannot tell you what the application's exit 1 means.

BREAK

```
kubectl run exit1 \
  --image=busybox:1.36 \
  --restart=Never \
  -- sh -c 'echo "Invalid configuration"; exit 1'
```

DIAGNOSE

```
kubectl describe pod exit1
```

Confirms:

```
Exit Code: 1
```

Then:

```
kubectl logs exit1
```

Confirms:

```
Invalid configuration
```

Teaching lesson

`describe` identifies the container lifecycle result.

`logs` identify the application-level cause.

Both are needed.

FIX

Correct whatever produced the application failure:

- command;
- config;
- Secret;
- environment;
- file permission;
- dependency.

COMBINE-OR-SPLIT

Combine with container exits immediately.

12. Init Container Failure

OVERVIEW

Init containers run before normal application containers.

Main containers do not begin until required init containers complete successfully.

ROOT CAUSES

1. init command fails
2. DB migration fails
3. DNS lookup fails
4. missing Secret
5. IAM denied
6. volume preparation fails

BREAK

```
initContainers:  
- name: migration  
  image: busybox:1.36  
  command:  
  - sh  
  - -c  
  - |  
    echo "Database migration failed"  
    exit 1
```

Explanation

The initialization stage terminates non-zero.

Kubernetes retries it.

The application container never begins.

DIAGNOSE

```
kubectl get pod <pod>
```

Possible:

```
Init:CrashLoopBackOff
```

Now:

```
kubectl describe pod <pod>
```

Most important:

```
kubectll logs <pod> -c migration
```

and:

```
kubectll logs <pod> -c migration --previous
```

Why `-c`?

The pod contains multiple containers.

We must explicitly ask for the failing init container.

FIX

Correct the init operation.

For dependencies, consider retries:

```
until nc -z database.example.internal 5432; do
  echo "Waiting for DB"
  sleep 5
done
```

But also set sensible timeouts rather than waiting forever.

VIDEO

EKS Init:CrashLoopBackOff - Diagnose Init Containers Before the App Starts

COMBINE-OR-SPLIT

Standalone.

13. Sidecar Container Failure

OVERVIEW

A pod can contain multiple containers.

The primary application may run correctly while:

- proxy;
- metrics exporter;
- log collector;
- security agent

fails.

BREAK

```
containers:
- name: web
  image: nginx

- name: sidecar
  image: busybox:1.36
  command:
  - sh
  - -c
  - exit 1
```

Why?

Both belong to the same pod.

The second container repeatedly terminates.

DIAGNOSE

```
kubectl get pod <pod>
```

Then inspect every container:

```
kubectl get pod <pod> \
-o jsonpath='{range .status.containerStatuses[*]}{.name}{ " => restarts="}
  {.restartCount}{ " ready="}{.ready}{ "\n"}{end}'
```

Then:

```
kubectl logs <pod> -c sidecar --previous
```

Teaching lesson

Never assume:

```
Pod problem = main application problem
```

Inspect all containers.

FIX

Correct only the failing sidecar.

Verify:

```
kubectl get pod <pod>
```

Expected:

2/2 Running

COMBINE-OR-SPLIT

Can pair with init containers as **multi-container pod troubleshooting**.

14. Application Works Locally but Fails in EKS

OVERVIEW

This is an excellent classroom topic because it teaches environmental differences.

Docker on a laptop does not reproduce:

- Kubernetes networking;
- Service discovery;
- AWS IAM;
- SecurityContext;
- probes;
- EKS VPC connectivity;
- Kubernetes resources;
- mounted configuration.

ROOT CAUSES

1. app binds 127.0.0.1
2. wrong Service targetPort
3. missing env variable
4. non-root permissions
5. read-only filesystem
6. AWS IAM difference
7. DNS difference
8. resource limit
9. network/security group

ROOT CAUSE - App Listens Only on localhost

BREAK

Configure application:

```
127.0.0.1:8080
```

Why does it work locally?

When you test inside the same container/process environment:

```
localhost -> application
```

works.

But another pod or Service connects using the pod's network interface IP.

The application isn't listening there.

DIAGNOSE

Inside container:

```
kubectl exec <pod> -- ss -lnt
```

Expected:

```
127.0.0.1:8080
```

Test locally:

```
kubectl exec <pod> -- \
  curl http://127.0.0.1:8080
```

Works.

From another pod:

```
kubectl run curltest \
  --rm -it \
  --image=curlimages/curl \
  --restart=Never \
  -- curl http://<pod-ip>:8080
```

Fails.

Now students have proof.

FIX

Bind application to:

```
0.0.0.0:8080
```

This means listen on all IPv4 interfaces.

ROOT CAUSE - AWS Credentials Different

DIAGNOSE

If image has AWS CLI:

```
kubectl exec <pod> -- aws sts get-caller-identity
```

Why?

It tells us what IAM identity the pod is actually using.

For modern EKS workloads/add-ons, Pod Identity can associate a Kubernetes ServiceAccount with an IAM role; IRSA remains supported.

Example Pod Identity role trust:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "pods.eks.amazonaws.com"
      },
      "Action": [
        "sts:AssumeRole",
        "sts:TagSession"
      ]
    }
  ]
}
```

Associate:

```
aws eks create-pod-identity-association \
  --cluster-name "$CLUSTER" \
  --namespace "$NS" \
  --service-account app-sa \
  --role-arn "arn:aws:iam::$ACCOUNT_ID:role/AppRole" \
  --region "$REGION"
```

Teaching lesson

Your laptop may use:

```
~/.aws/credentials
```

while a production pod should use workload identity.

VIDEO

Works on My Laptop but Fails on EKS - 9 Differences to Troubleshoot

COMBINE-OR-SPLIT

Standalone flagship.

15. Application Starts but Pod Never Becomes Ready

OVERVIEW

Teach the distinction:

Running = process exists

Ready = Kubernetes considers pod eligible for Service traffic

A pod can therefore show:

Running 0/1

ROOT CAUSES

1. readiness path wrong
2. readiness port wrong
3. dependency unavailable
4. probe timeout
5. sidecar unready
6. readiness gate

BREAK

```
readinessProbe:
  httpGet:
    path: /does-not-exist
    port: 80
    periodSeconds: 3
```

What happens?

Nginx continues running.

Kubernetes probes a path returning 404.

The pod isn't killed because this is **readiness**, not liveness.

But it remains:

```
Ready=False
```

DIAGNOSE

```
kubectl get pod <pod>
```

Then:

```
kubectl describe pod <pod>
```

Look for:

```
Readiness probe failed:  
HTTP probe failed with statuscode: 404
```

Check conditions:

```
kubectl get pod <pod> \\  
-o jsonpath='{.status.conditions}'
```

Check Service endpoints:

```
kubectl get endpointslice \\  
-l kubernetes.io/service-name=<service>
```

Why EndpointSlice?

It proves the impact of readiness.

Unready backends may not be used for normal Service traffic.

FIX

Correct endpoint:

```
readinessProbe:  
  httpGet:  
    path: /  
    port: 80
```

Verify:

```
kubectl get pod <pod> -w
```

Expected:

Section 2 - Scheduling Problems

16. 0/x nodes are available

OVERVIEW

This isn't one error.

It's a scheduler summary containing the root cause.

Examples:

```
0/3 nodes are available:
3 Insufficient cpu
```

or:

```
0/3 nodes are available:
1 node had untolerated taint,
2 didn't match Pod's node affinity
```

Teach students to read the **whole sentence**.

ROOT CAUSES

1. CPU
2. memory
3. selectors
4. affinity
5. taints
6. anti-affinity
7. topology spread
8. PVC topology
9. too many pods

BREAK

Use multiple pods from Section 1 that intentionally produce each reason.

DIAGNOSE

```
kubectrl describe pod <pending-pod>
```

Why?

Scheduler Events identify rejected constraints.

Also:

```
kubectl get events \
  --field-selector reason=FailedScheduling \
  --sort-by='.lastTimestamp'
```

What does this provide?

A chronological list of scheduling failures, useful when pod status alone is insufficient.

FIX

Never apply a generic fix.

Map scheduler message -> constraint:

```
Insufficient cpu
-> request/capacity

untolerated taint
-> toleration/node eligibility

didn't match affinity
-> labels/affinity

Too many pods
-> pod networking/capacity

volume node affinity conflict
-> storage topology
```

VIDEO

0/3 Nodes Are Available on EKS - Learn to Read Scheduler Errors

COMBINE-OR-SPLIT

Flagship umbrella scheduling lesson.

17. Insufficient CPU

Use the detailed CPU Pending lab above.

EXTRA TEACHING DEPTH

Show:

```
kubectl top nodes
```

versus:

```
kubectl describe nodes
```

Explain with a classroom example:

Node:

```
4 CPU cores
```

Pods request:

```
Pod A = 1500m  
Pod B = 1500m  
System = reserved capacity
```

A new pod requests:

```
2000m
```

Even if current actual application usage is low, scheduling may fail because Kubernetes promises requested resources.

VIDEO

Why EKS Says Insufficient CPU When CPU Usage Is Only 20%

COMBINE-OR-SPLIT

Standalone because request vs utilization is foundational.

18. Insufficient Memory

Same mechanics as CPU scheduling.

BREAK

```
kubectl run memory-scheduler-demo \\\n  --image=nginx \\\n  --requests='memory=1000Gi,cpu=10m'
```

DIAGNOSE

```
kubectl describe pod memory-scheduler-demo
```

Look for:

```
Insufficient memory
```

TEACHER COMPARISON

Symptom	Stage
Insufficient memory	Before scheduling
MemoryPressure	Node running low on memory
OOMKilled	Running container killed

FIX

Right-size requests or add appropriate compute capacity.

COMBINE-OR-SPLIT

Combine with CPU scheduling as one resource-fit video, or use independently for classroom exercises.

19. NodeSelector Mismatch

BREAK

```
nodeSelector:  
  workload: payment
```

with no matching node.

DIAGNOSE

```
kubectl describe pod <pod>
```

then:

```
kubectl get nodes -L workload
```

Why both?

First command tells us:

```
scheduler rejected nodes due to selector
```

Second determines:

what node labels actually exist

FIX

Either correct the requirement or configure nodes consistently.

For managed EKS node groups, persist labels through managed-node-group configuration instead of relying only on manual node mutation.

VIDEO

EKS NodeSelector Troubleshooting - Pod Pending Even Though Nodes Are Ready

COMBINE-OR-SPLIT

Combine with node affinity.

20. Node Affinity Prevents Scheduling

OVERVIEW

Node affinity is more expressive than `nodeSelector`.

Important distinction:

`requiredDuringSchedulingIgnoredDuringExecution`

is mandatory.

`preferredDuringSchedulingIgnoredDuringExecution`

is a preference.

BREAK

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: eks.amazonaws.com/nodegroup
              operator: In
              values:
                - nodegroup-does-not-exist
```

Why?

We're requiring a managed node group label value that no node possesses.

DIAGNOSE

```
kubectl describe pod <pod>
```

Look for affinity/selector mismatch.

Then:

```
kubectl get nodes \
-L eks.amazonaws.com/nodegroup
```

Now students can visually compare required vs available values.

FIX

Change to the real node group:

```
values:
- workers
```

or make it preferred if hard placement isn't required.

GOTCHA

Cluster Autoscaler can't invent a completely nonexistent matching node-group configuration just because a pod has impossible affinity.

COMBINE-OR-SPLIT

With NodeSelector.

21. Pod Anti-Affinity Problems

OVERVIEW

Anti-affinity says:

Keep certain pods away from each other.

Useful for HA.

Dangerous when constraints are stricter than cluster capacity.

BREAK

Suppose two worker nodes.

Create three replicas with:

```
podAntiAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
  - labelSelector:
      matchLabels:
        app: anti-demo
      topologyKey: kubernetes.io/hostname
```

Why will replica 3 fail?

Rule says:

```
Only one matching pod per hostname
```

There are only two eligible hostnames.

Replica three has nowhere valid to run.

DIAGNOSE

```
kubectl get pods -l app=anti-demo -o wide
```

Then:

```
kubectl describe pod <pending-pod>
```

Inspect nodes:

```
kubectl get nodes \
  -L kubernetes.io/hostname
```

FIX

Options:

- add nodes;
- reduce replicas;
- change required anti-affinity to preferred;
- change topology domain.

Example:

```
preferredDuringSchedulingIgnoredDuringExecution:
```

Why?

Scheduler tries to spread replicas but can violate preference when required to maintain availability.

COMBINE-OR-SPLIT

Combine with topology spread constraints.

22. Taints and Tolerations

Use the earlier lab but teach all effects.

TAINT EFFECTS

```
NoSchedule
```

New non-tolerating pods aren't scheduled.

```
PreferNoSchedule
```

Scheduler tries to avoid the node.

```
NoExecute
```

Can also evict existing non-tolerating pods.

BREAK

```
kubectl taint nodes "$NODE" \
  workload=special:NoSchedule
```

DIAGNOSE

```
kubectl describe node "$NODE" | grep -A2 Taints
```

and:

```
kubectl describe pod <pod>
```

FIX

```
tolerations:
- key: workload
  operator: Equal
  value: special
  effect: NoSchedule
```

IMPORTANT CLASSROOM POINT

A toleration answers:

May I use this node?

Node affinity answers:

Which nodes do I want?

They are often used together.

COMBINE-OR-SPLIT

Standalone.

23. Pod Topology Spread Constraints

OVERVIEW

Topology spread constraints try to distribute replicas across topology domains such as:

nodes
Availability Zones

BREAK

```
topologySpreadConstraints:  
- maxSkew: 1  
  topologyKey: topology.kubernetes.io/zone  
  whenUnsatisfiable: DoNotSchedule  
  labelSelector:  
    matchLabels:  
      app: frontend
```

Create a cluster with insufficient eligible capacity across required zones.

Why?

`DoNotSchedule` makes the rule hard.

If placing a pod violates allowed skew, it remains Pending.

DIAGNOSE

```
kubectl describe pod <pending-pod>
```

Then:

```
kubectl get nodes \
  -L topology.kubernetes.io/zone
```

and:

```
kubectl get pods -l app=frontend -o wide
```

This lets students visualize replica distribution.

FIX

Options:

- add capacity in missing AZ;
- loosen `maxSkew`;
- use `ScheduleAnyway`;
- correct topology labels.

COMBINE-OR-SPLIT

Combine with anti-affinity as:

High Availability Scheduling That Accidentally Prevents Scheduling

24. PVC Prevents Scheduling

OVERVIEW

Storage can influence scheduler node choice.

EBS volumes are tied to Availability Zones, making this especially useful on EKS.

ROOT CAUSES

1. PVC unbound
2. StorageClass absent
3. EBS topology mismatch
4. CSI failure
5. incompatible access mode
6. binding mode issues

DIAGNOSE

```
kubectl describe pod <pod>
kubectl get pvc,pv
kubectl describe pvc <pvc>
kubectl get storageclass -o yaml
```

For bound EBS:

```
kubectl get pv <pv> \
-o jsonpath='{.spec.nodeAffinity}{"\n"}'
```

Compare worker zones:

```
kubectl get nodes \
-L topology.kubernetes.io/zone
```

Teaching explanation

If volume exists in:

```
ap-south-1a
```

the pod cannot simply attach it to an EC2 worker in:

```
ap-south-1b
```

FIX

For dynamic EBS provisioning, a StorageClass using:

```
volumeBindingMode: WaitForFirstConsumer
```

allows volume topology to be selected with awareness of the consuming pod's placement.

COMBINE-OR-SPLIT

Combine with PVC Pending in Storage.

25. Too Many Pods on Node

OVERVIEW

A node can have:

```
free CPU
free memory
```

and still reject additional pods.

EKS pod density may be constrained by kubelet limits and, in traditional VPC CNI configurations, underlying networking capacity. The VPC CNI creates/attaches ENIs and assigns VPC addresses to pods on EC2 nodes.

DIAGNOSE

```
kubectl get nodes \
  -o custom-columns=NAME:.metadata.name,MAXPODS:.status.allocatable.pods
```

Count:

```
kubectl get pods -A \
  --field-selector spec.nodeName="$NODE" \
  --no-headers | wc -l
```

Pending pod:

```
kubectl describe pod <pod>
```

Look for:

```
Too many pods
```

FIX

Depending on architecture:

- additional workers;
- larger networking-capable instance types;
- prefix delegation;
- subnet IP planning;
- lower pod density.

VIDEO

EKS Too Many Pods - Why CPU Is Free but Scheduler Still Refuses Pods

COMBINE-OR-SPLIT

Standalone high-value EKS topic.

Section 3 - Probes

26. Liveness Probe Failing

OVERVIEW

Liveness answers:

```
Should Kubernetes restart this container?
```

Therefore a bad liveness design can cause an outage.

ROOT CAUSES

1. wrong path
2. wrong port
3. timeout
4. initial delay too short
5. application deadlock
6. external dependency incorrectly included

BREAK

```
livenessProbe:  
  httpGet:  
    path: /wrong-health  
    port: 80  
  periodSeconds: 5  
  failureThreshold: 1
```

EXPLANATION

Nginx returns 404.

Kubelet interprets failure as application death and restarts the container.

DIAGNOSE

```
kubectl describe pod <pod>
```

Look for:

```
Liveness probe failed
```

then:

Killing container

Manual reproduction:

```
kubectl exec <pod> -- \
  wget -S -O- http://127.0.0.1/wrong-health
```

FIX

Correct path.

More importantly teach:

A liveness endpoint should normally answer:

Is this process internally alive enough that restarting is better than leaving it running?

It should not fail merely because an optional remote service is briefly unavailable.

COMBINE-OR-SPLIT

Standalone.

27. Readiness Probe Failing

Readiness asks:

Should this pod receive traffic?

BREAK

```
readinessProbe:
  httpGet:
    path: /bad-ready
    port: 80
```

DIAGNOSE

```
kubectl describe pod <pod>
```

Then:

```
kubectl get endpointslice \
  -l kubernetes.io/service-name=<service> \
  -o yaml
```

Why?

We want students to see the relationship:

```
Probe fails
↓
Pod not Ready
↓
Backend no longer eligible normally
↓
Service traffic affected
```

FIX

Correct endpoint/dependency/timing.

COMBINE-OR-SPLIT

Combine with “Running but not Ready.”

28. Startup Probe Failing

OVERVIEW

Startup probes are designed for applications that legitimately need longer initialization.

While startup probe has not succeeded, liveness/readiness behavior is gated appropriately.

BREAK

```
startupProbe:
  httpGet:
    path: /never-exists
    port: 80
  periodSeconds: 2
  failureThreshold: 3
```

Time budget

2 seconds x 3 failures ~= 6 seconds

Then startup is considered failed.

DIAGNOSE

```
kubectl describe pod <pod>
```

Look for:

```
Startup probe failed
```

and container restart.

FIX

Correct endpoint and calculate realistic startup budget.

If maximum startup is two minutes:

```
periodSeconds: 5  
failureThreshold: 24
```

gives approximately:

```
120 seconds
```

COMBINE-OR-SPLIT

Combine with application startup too slow.

29. HTTP Probe Returning 404/500

OVERVIEW

404 and 500 provide different evidence.

```
404 -> server reached; route/resource not found  
500 -> server reached; application encountered internal failure
```

Neither automatically means Kubernetes networking is broken.

BREAK

Use:

```
readinessProbe:  
  httpGet:  
    path: /does-not-exist  
    port: 80
```

DIAGNOSE

```
kubectl describe pod <pod>
```

Manually:

```
kubectl exec <pod> -- \
  curl -i http://127.0.0.1:80/does-not-exist
```

Why curl manually?

It separates:

Kubernetes probe mechanism

from:

application HTTP behavior

FIX

404:

- correct path;
- routing configuration.

500:

- investigate app logs/dependencies.

```
kubectl logs <pod>
```

COMBINE-OR-SPLIT

Combine wrong path + wrong port + HTTP statuses.

30. Probe Timeout

OVERVIEW

A timeout means the target did not return an acceptable result before `timeoutSeconds`.

ROOT CAUSES

1. slow handler
2. CPU throttling
3. thread starvation

4. downstream API delay
5. DB delay
6. timeout too low

DIAGNOSE

```
kubectl describe pod <pod>
```

Then manually time endpoint:

```
kubectl exec <pod> -- \
time curl -s http://127.0.0.1:8080/health
```

CPU:

```
kubectl top pod <pod> --containers
```

Logs:

```
kubectl logs <pod>
```

Teaching point

Do not immediately increase timeout.

First ask:

Why did a health endpoint become slow?

FIX

Fix root latency.

Then choose realistic:

```
timeoutSeconds: 3
```

based on measured application behavior.

COMBINE-OR-SPLIT

Pair with CPU throttling.

31. Incorrect Probe Port

BREAK

Nginx listens:

```
80
```

Set:

```
livenessProbe:
  httpGet:
    path: /
    port: 8080
```

DIAGNOSE

```
kubectl describe pod <pod>
```

likely:

```
connection refused
```

Check listener:

```
kubectl exec <pod> -- ss -lnt
```

Why?

`ss` tells us which ports actually have listening sockets.

FIX

```
port: 80
```

Teaching point

Do not confuse:

```
Service port
targetPort
container listener
probe port
```

32. Application Startup Too Slow

BREAK

Make application wait:

```
sh -c 'sleep 60; exec nginx -g "daemon off;"'
```

Configure liveness to begin in 5 seconds.

WHY IT BREAKS

Application is behaving exactly as designed-it simply isn't ready yet.

Liveness repeatedly kills it before reaching second 60.

Therefore it can **never finish starting**.

DIAGNOSE

```
kubectl describe pod <pod>
```

Look for probe kills shortly after each start.

```
kubectl logs <pod> --previous
```

Look at timestamps.

FIX

Use startupProbe.

```
startupProbe:
  httpGet:
    path: /
    port: 80
  periodSeconds: 5
  failureThreshold: 24
```

33. Readiness Probe 504

OVERVIEW

HTTP 504 generally means:

```
a gateway/proxy was reached
but it timed out waiting for an upstream
```

That's very different from connection refused.

DIAGNOSE

```
kubectl exec <pod> -- \
  curl -sv http://127.0.0.1:8080/ready
```

Then all container logs:

```
kubectl logs <pod> --all-containers
```

If proxy sidecar exists:

```
kubectl logs <pod> -c <proxy>
```

Test upstream directly:

```
kubectl exec <pod> -- \
  nc -vz <upstream-host> <port>
```

FIX

Repair the slow/unreachable upstream or redesign readiness to depend only on systems required to serve traffic.

COMBINE-OR-SPLIT

Standalone incident-style video.

34. Liveness Probe Causing Restart Loops

This combines CrashLoop + probes.

DIAGNOSE SEQUENCE

```
kubectl get pod <pod> -w
```

then:

```
kubectl describe pod <pod>
```

then:

```
kubectl logs <pod> --previous
```

Build the evidence:

```
restartCount rising
+
Liveness probe failed
+
Killing container
```

This proves kubelet is initiating the restart.

FIX

Correct:

- path;
- timeout;
- startup period;
- failure threshold;
- dependency design.

COMBINE-OR-SPLIT

Combine with Liveness Probe Failing.

Section 4 - Networking

35. Pod Cannot Communicate With Another Pod

OVERVIEW

Troubleshoot network layers instead of immediately blaming “Kubernetes networking.”

Sequence:

```
Destination process listening?
↓
Correct pod IP?
↓
TCP port?
↓
NetworkPolicy?
↓
CNI?
↓
VPC networking?
```

BREAK LAB

```
kubectl create deployment net-web \
  --image=nginx

kubectl expose deployment net-web \
  --port=80

kubectl run net-client \
  --image=curlimages/curl \
  --restart=Never \
  -- sleep 3600
```

Find backend:

```
POD_IP=$(kubectl get pod \
  -l app=net-web \
  -o jsonpath='{.items[0].status.podIP}')
```

Test:

```
kubectl exec net-client -- \
  curl -v "http://$POD_IP:80"
```

ROOT CAUSES

1. app bound localhost
2. incorrect port
3. NetworkPolicy
4. CNI
5. pod security groups
6. node/VPC routing
7. app isn't listening

DIAGNOSE

Destination:

```
kubectl exec <destination-pod> -- ss -lnt
```

Why?

Before investigating AWS, prove an application is actually listening.

Then test TCP:

```
kubectl exec net-client -- \
  nc -vz "$POD_IP" 80
```

Then CNI:

```
kubectl -n kube-system get pods \
-l k8s-app=aws-node
```

The Amazon VPC CNI provides pod VPC networking on EC2-backed EKS and allocates pod addresses through VPC networking resources.

FIX

Repair the first failed layer rather than reinstalling networking blindly.

COMBINE-OR-SPLIT

Standalone networking foundation.

36. Cross-Namespace Communication Fails

OVERVIEW

Namespaces themselves don't automatically isolate networking.

Most failures are:

- DNS naming;
- NetworkPolicy;
- Service reference mistakes.

BREAK

Service:

```
api
```

namespace:

```
backend
```

From namespace `frontend`, try:

```
curl http://api
```

Short names normally search the caller's namespace first.

DIAGNOSE

Use:

```
nslookup api.backend.svc.cluster.local
```

Then:

```
curl http://api.backend.svc.cluster.local
```

Why?

We're proving whether the problem is:

```
DNS naming
```

rather than packet connectivity.

Check policies:

```
kubectl get networkpolicy -A
```

FIX

Use:

```
api.backend
```

or:

```
api.backend.svc.cluster.local
```

and correct namespace policies.

37. Service Not Reachable

OVERVIEW

Use this exact diagnostic ladder:

```
Service object
↓
Selector
↓
EndpointSlice
↓
Pod readiness
↓
Pod IP
↓
targetPort
↓
kube-proxy/service dataplane
```

DIAGNOSE

```
kubectl get svc <service> -o yaml
```

Why?

Check:

- selector;
- port;
- targetPort;
- ClusterIP.

Then:

```
kubectl get pods --show-labels
```

Compare selector.

Then:

```
kubectl get endpointslice \
-l kubernetes.io/service-name=<service>
```

Why EndpointSlice?

It tells us which pod IPs Kubernetes considers service backends.

Test backend directly:

```
curl http://<pod-ip>:<target-port>
```

Then test Service:

```
curl http://<cluster-ip>:<service-port>
```

Interpretation

```
Pod IP fails
-> backend/application/network

Pod IP works + Service IP fails
-> Service/kube-proxy path becomes likely
```

COMBINE-OR-SPLIT

Standalone flagship networking topic.

38. ClusterIP Not Working

BREAK

First create a valid Service and prove it works.

Then create an incorrect targetPort.

```
ports:
- port: 80
  targetPort: 9999
```

DIAGNOSE

```
kubectl get svc <svc> -o yaml
kubectl get endpointslice \
-l kubernetes.io/service-name=<svc>
```

Direct:

```
curl http://<pod-ip>:80
```

Service:

```
curl http://<cluster-ip>:80
```

If direct pod path works but Service IP doesn't, investigate service translation/kube-proxy.

```
kubectl -n kube-system get ds kube-proxy
```

FIX

Correct targetPort or restore kube-proxy depending on evidence.

COMBINE-OR-SPLIT

Combine with kube-proxy.

39. DNS Resolution Failure

OVERVIEW

CoreDNS normally provides Kubernetes cluster DNS.

A timeout and NXDOMAIN mean different things.

NXDOMAIN

-> DNS answered but name doesn't exist

timeout

-> DNS communication/server problem

BREAK/TEST

```
kubectl run dns-test \
  --image=busybox:1.36 \
  --restart=Never \
  -- sleep 3600
```

```
kubectl exec dns-test -- \
  nslookup kubernetes.default.svc.cluster.local
```

DIAGNOSE

Check resolver configuration:

```
kubectl exec dns-test -- cat /etc/resolv.conf
```

Check CoreDNS:

```
kubectl -n kube-system get deploy coredns
kubectl -n kube-system get pods -l k8s-app=kube-dns
```

Logs:

```
kubectl -n kube-system logs deployment/coredns \
  --tail=200
```

Service:

```
kubectl -n kube-system get svc kube-dns
```

FIX

Depending on evidence:

- restore CoreDNS;
- fix NetworkPolicy UDP/TCP 53;
- repair Corefile;
- fix upstream/VPC DNS;
- increase DNS capacity.

COMBINE-OR-SPLIT

Standalone flagship.

40. CoreDNS Failure

ROOT CAUSES

1. pod Pending
2. CrashLoopBackOff
3. bad Corefile
4. resources exhausted
5. scheduling constraints
6. EKS add-on/version issue

DIAGNOSE

```
kubectl -n kube-system get deployment coredns
kubectl -n kube-system get pods \
  -l k8s-app=kube-dns -o wide
```

If Pending:

```
kubectl -n kube-system describe pod <coredns-pod>
```

If running/crashing:

```
kubectl -n kube-system logs deployment/coredns
```

Configuration:

```
kubectl -n kube-system get cm coredns -o yaml
```

EKS:

```
aws eks describe-addon \
  --cluster-name "$CLUSTER" \
  --addon-name coredns \
  --region "$REGION"
```

Why AWS command?

It distinguishes:

```
self-managed Kubernetes object
```

from:

```
EKS-managed add-on lifecycle
```

FIX

Use a compatible EKS add-on version rather than randomly installing upstream CoreDNS.

41. nslookup Works but Application Doesn't

OVERVIEW

This teaches OSI/application-layer reasoning.

DNS working proves:

```
hostname -> address
```

It does not prove:

- destination port open;
- TLS valid;
- authentication;
- proxy settings;
- application behavior.

DIAGNOSE

```
nslookup api.example.com
```

Then:

```
nc -vz api.example.com 443
```

Then:

```
curl -vk https://api.example.com
```

Each answers a different question

nslookup:

```
Can I resolve it?
```

nc:

Can TCP connect?

curl:

Can the application protocol/TLS exchange complete?

Inspect proxy configuration:

```
kubectl exec <pod> -- env | grep -i proxy
```

COMBINE-OR-SPLIT

Standalone. Excellent classroom troubleshooting lesson.

42. NetworkPolicy Blocking Traffic

BREAK

Apply a namespace-wide deny:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: breakfix
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

What does this do?

It selects every pod and declares both ingress and egress policy enforcement.

Without allow rules, selected traffic is denied by the active NetworkPolicy implementation.

On EKS, VPC CNI NetworkPolicy support has specific platform/mode considerations; don't assume every Fargate/Windows setup behaves identically.

DIAGNOSE

```
kubectl get networkpolicy
kubectl describe networkpolicy deny-all
```

Retry:

```
kubectl exec net-client -- \
  curl http://net-web
```

DNS may also fail because outbound DNS was denied.

Test:

```
kubectl exec net-client -- \
  nslookup kubernetes.default
```

FIX

Allow DNS and required application flows explicitly.

Teaching point:

Default deny is good security only when required communication paths are intentionally permitted.

43. Ingress Returns 404

OVERVIEW

With AWS Load Balancer Controller/ALB, distinguish:

```
ALB rule 404
```

from:

```
backend application 404
```

ROOT CAUSES

1. host mismatch
2. path mismatch
3. backend route absent
4. incorrect IngressClass
5. stale/misconfigured ALB rule

DIAGNOSE

```
kubectl describe ingress <ingress>
```

Why?

It displays:

- rules;
- backend;
- controller Events;
- ALB address.

Test hostname explicitly:

```
curl -v \
-H 'Host: app.example.com' \
http://<alb-dns>/expected-path
```

Then backend directly:

```
kubectl run curl-test \
--rm -it \
--image=curlimages/curl \
--restart=Never \
-- curl -v http://<service>:<port>/expected-path
```

Interpretation

Backend itself returns 404:

```
application route problem
```

Backend works but ALB route returns 404:

```
Ingress host/path/rule
```

COMBINE-OR-SPLIT

Standalone 404 routing lesson.

44. Ingress Returns 502

OVERVIEW

A 502 normally indicates the proxy/load balancer couldn't complete a valid exchange with a selected backend.

ROOT CAUSES

1. wrong backend port
2. HTTP/HTTPS mismatch
3. backend resets connection
4. malformed response
5. sidecar/proxy problem
6. target connectivity

DIAGNOSE

Kubernetes first:

```
kubectl get svc,endsliceslice
kubectl describe ingress <ingress>
```

Then AWS target health:

```
aws elbv2 describe-target-health \
  --target-group-arn "$TG_ARN" \
  --region "$REGION"
```

Why check target group?

The ALB's view of backend health may differ from Kubernetes readiness.

Direct backend:

```
kubectl run curltest \
  --rm -it \
  --image=curlimages/curl \
  --restart=Never \
  -- curl -sv http://<service>:<port>
```

FIX

Correct:

- targetPort;
- backend protocol;
- listener;
- application response behavior.

COMBINE-OR-SPLIT

Combine with 503.

45. Ingress Returns 503

OVERVIEW

503 commonly points toward no usable healthy backend or application unavailability.

ROOT CAUSES

1. zero endpoints
2. readiness failing
3. ALB targets unhealthy
4. health-check path wrong
5. Service selector wrong
6. deployment rollout leaves no ready pods

DIAGNOSE

```
kubectl get pods
kubectl get endpointslice \
  -l kubernetes.io/service-name=<service>
```

Then AWS:

```
aws elbv2 describe-target-health \
  --target-group-arn "$TG_ARN" \
  --region "$REGION"
```

Teaching chain

```
Pod readiness
↓
Service endpoints
↓
ALB target health
↓
Client response
```

FIX

Fix the first failing layer.

46. LoadBalancer Stuck Pending

OVERVIEW

The Kubernetes Service exists, but cloud load-balancer provisioning hasn't completed.

ROOT CAUSES

1. controller missing
2. AWS IAM denied
3. subnet discovery
4. invalid annotations
5. quota
6. AWS resource creation error

DIAGNOSE

```
kubectl describe service <service>
```

Why?

Cloud controller/controller errors appear as Events.

Then:

```
kubectl -n kube-system get deployment \
aws-load-balancer-controller
```

Logs:

```
kubectl -n kube-system logs \
deployment/aws-load-balancer-controller \
--tail=200
```

Look for:

```
AccessDenied
```

or subnet-discovery/resource messages.

IAM TEACHING

If using EKS Pod Identity for a controller/workload, the IAM role's trust relationship uses:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "pods.eks.amazonaws.com"
      },
      "Action": [
        "sts:AssumeRole",
        "sts:TagSession"
      ]
    }
  ]
}
```

Then associate the controller's Kubernetes ServiceAccount with the IAM role.

Explain:

```
Kubernetes ServiceAccount
    ↓
Pod Identity association
    ↓
IAM Role
    ↓
AWS API permissions
```

AWS recommends Pod Identity for add-on/workload IAM where supported, while IRSA is still available.

COMBINE-OR-SPLIT

Standalone.

47. Wrong Service Selector

BREAK

Pods:

```
labels:
  app: frontend
```

Service:

```
selector:
  app: frontned
```

Notice typo.

DIAGNOSE

```
kubectl get svc <service> -o yaml
```

Then:

```
kubectl get pods --show-labels
```

Then:

```
kubectl get endpointslice \
  -l kubernetes.io/service-name=<service>
```

Expected no backend endpoints.

Why three commands?

They prove:

```
Service selector
does not match
Pod labels
therefore
EndpointSlice empty
```

FIX

Correct:

```
selector:
  app: frontend
```

Verify EndpointSlice repopulates.

COMBINE-OR-SPLIT

Combine with Service has no endpoints.

48. Service Has No Endpoints

ROOT CAUSES

1. selector mismatch
2. no matching pods
3. pods not Ready
4. wrong namespace
5. manual/headless service design

DIAGNOSE

```
kubectl get endpointslice \
  -l kubernetes.io/service-name=<service> \
  -o wide
```

Then:

```
kubectl get pods --show-labels
```

and:

```
kubectl get pods \
  -o custom-columns=NAME:.metadata.name,READY:.status.conditions[-1].status
```

Better inspect exact readiness via:

```
kubectl describe pod <pod>
```

TEACHING POINT

“No endpoints” is usually not a mysterious network packet problem.
Often Kubernetes simply has no eligible backend.

49. TargetPort / Port Mismatch

BREAK

Nginx:

```
80
```

Service:

```
ports:
- port: 80
  targetPort: 8080
```

Meaning

Clients send to Service:

```
80
```

Kubernetes forwards to backend:

```
8080
```

But nginx isn't listening there.

DIAGNOSE

```
kubectl get svc <service> -o yaml
```

Then:

```
kubectl exec <pod> -- ss -lnt
```

Compare:

```
Service targetPort = 8080  
App listener = 80
```

Root cause proven.

FIX

```
targetPort: 80
```

50. External Service Inaccessible

OVERVIEW

Examples:

- RDS;
- external HTTPS API;
- another VPC;
- corporate network.

ROOT CAUSES

1. DNS
2. security group
3. no NAT/endpoint
4. route table
5. NACL
6. NetworkPolicy
7. proxy

8. TLS

DIAGNOSE LADDER

DNS:

```
nslookup "$HOST"
```

TCP:

```
nc -vz "$HOST" "$PORT"
```

Application:

```
curl -vk "https://$HOST"
```

Why this order?

Each step depends on the previous layer.

Don't start with HTTP if DNS itself fails.

For RDS, test:

```
nc -vz "$RDS_HOST" 5432
```

Then investigate RDS SG inbound source and EKS pod/node networking architecture.

COMBINE-OR-SPLIT

Standalone.

51. kube-proxy Issue

OVERVIEW

On conventional EC2-based EKS, kube-proxy participates in Kubernetes Service connectivity.

A useful symptom:

```
Direct Pod IP works  
ClusterIP does not
```

BREAK SAFELY

Do not break production.

On disposable cluster, intentionally make kube-proxy unschedulable by applying an impossible nodeSelector to the DaemonSet, after exporting the original manifest for recovery:

```
kubectl -n kube-system get ds kube-proxy \
-o yaml > /tmp/kube-proxy-backup.yaml
```

Then patch lab environment only.

DIAGNOSE

```
kubectl -n kube-system get ds kube-proxy
```

Then:

```
kubectl -n kube-system get pods \
-l k8s-app=kube-proxy -o wide
```

Logs:

```
kubectl -n kube-system logs ds/kube-proxy \
--tail=200
```

Test:

```
pod IP -> works
ClusterIP -> fails
```

This isolates Service dataplane behavior.

FIX

Restore:

```
kubectl apply -f /tmp/kube-proxy-backup.yaml
```

For EKS-managed add-on installations, manage compatible versions through EKS add-on lifecycle rather than replacing them arbitrarily.

52. CNI Plugin Issue

OVERVIEW

This is one of your strongest EKS-specific topics.

The Amazon VPC CNI on EC2 EKS nodes manages ENIs and pod VPC addresses.

ROOT CAUSES

1. `aws-node` unhealthy
2. IAM denied
3. subnet exhausted
4. ENI limits
5. configuration error
6. incompatible add-on

DIAGNOSE

```
kubectl -n kube-system get ds aws-node
```

Why?

A DaemonSet should provide CNI presence across relevant workers.

Then:

```
kubectl -n kube-system get pods \
-l k8s-app=aws-node \
-o wide
```

Logs:

```
kubectl -n kube-system logs \
ds/aws-node \
-c aws-node \
--tail=200
```

Pod:

```
kubectl describe pod <affected-pod>
```

Look for:

```
FailedCreatePodSandBox
failed to assign an IP address
```

AWS:

```
aws eks describe-addon \
  --cluster-name "$CLUSTER" \
  --addon-name vpc-cni \
  --region "$REGION"
```

Subnet IPs:

```
aws ec2 describe-subnets \
  --subnet-ids $SUBNETS \
  --query 'Subnets[].{ID:SubnetId,FreeIPs:AvailableIpAddressCount}'
```

FIX

Correct the evidence-specific cause:

- IAM association;
- subnet capacity;
- add-on health/version;
- prefix delegation;
- EC2 networking capacity.

COMBINE-OR-SPLIT

Standalone flagship.

Section 5 - Storage

53. PVC Stuck Pending

OVERVIEW

PVC Pending means Kubernetes hasn't obtained suitable persistent storage.

On EKS, EBS CSI is a core troubleshooting layer. AWS recommends using the EKS EBS CSI add-on; EBS cannot be mounted directly to Fargate pods.

ROOT CAUSES

1. StorageClass missing
2. CSI driver absent
3. CSI IAM denied
4. WaitForFirstConsumer
5. access mode issue

6. AWS EBS/KMS/quota failure

BREAK - Missing StorageClass

```
storageClassName: missing-class
```

DIAGNOSE

```
kubectl get pvc
```

Then:

```
kubectl describe pvc <pvc>
```

Why?

PVC Events expose provisioner/binding errors.

Then:

```
kubectl get sc
```

Compare requested vs existing.

BREAK - EBS CSI IAM Failure

The CSI controller must call AWS APIs to provision EBS.

Without permission, PVC Events can report `UnauthorizedOperation`.

DIAGNOSE

```
kubectl -n kube-system get pods \
-l app.kubernetes.io/name=aws-ebs-csi-driver
```

Controller logs:

```
kubectl -n kube-system logs \
deployment/ebs-csi-controller \
-c ebs-plugin \
--tail=200
```

FIX

Use the EKS-managed EBS CSI add-on and the currently appropriate scoped IAM policy/Pod Identity configuration for the driver.

Verify:

```
kubectl get pvc -w
```

Expected:

```
Pending -> Bound
```

54. PV Binding Failure

ROOT CAUSES

1. capacity mismatch
2. access mode
3. StorageClass mismatch
4. selector
5. existing claimRef
6. topology

BREAK

Create a static PV of:

```
1Gi
```

PVC asks:

```
10Gi
```

Why?

A 1Gi volume cannot satisfy a 10Gi claim.

DIAGNOSE

```
kubectl get pv
kubectl get pvc
kubectl describe pv <pv>
kubectl describe pvc <pvc>
```

Compare:

```
Capacity
AccessModes
StorageClass
Status
Claim
```

FIX

Make requirements compatible or use correct dynamic provisioning.

55. StorageClass Missing

BREAK

```
storageClassName: class404
```

DIAGNOSE

```
kubectl get sc
kubectl describe pvc <pvc>
```

Teacher explanation

StorageClass tells Kubernetes **how storage should be dynamically provisioned**.

If the named class doesn't exist, Kubernetes has no matching provisioning policy.

FIX

Use correct EBS CSI StorageClass.

Important EKS distinction: EKS Auto Mode has different integrated storage behavior/provisioner conventions than a conventional EBS CSI deployment.

56. Wrong Access Mode

OVERVIEW

Access modes describe how Kubernetes expects the volume to be mounted.

Common:

```
ReadWriteOnce
ReadOnlyMany
ReadWriteMany
ReadWriteOncePod
```

BREAK

Request a storage pattern unsupported by the selected backend/class.

Example classroom discussion:

```
Multiple replicas across nodes
+
single EBS volume
+
shared-writable expectation
```

DIAGNOSE

```
kubectl describe pvc <pvc>
kubectl get pv <pv> -o yaml
```

Check:

```
accessModes
```

FIX

For stateful replicas:

```
one EBS PVC per replica
```

often using StatefulSet `volumeClaimTemplates`.

For true shared filesystem requirements consider EFS rather than pretending one normal EBS attachment is a shared network filesystem.

57. Volume Mount Failure

OVERVIEW

Provisioning, attaching and mounting are separate stages.

Teach:

```
Provision
↓
Attach
↓
Mount
↓
Application uses filesystem
```

ROOT CAUSES

1. filesystem
2. invalid mount options
3. CSI node plugin
4. permission

5. EFS networking
6. corrupted filesystem

DIAGNOSE

```
kubectl describe pod <pod>
```

Look for mount event.

Then:

```
kubectl get volumeattachment
```

Check CSI pods on the node:

```
kubectl -n kube-system get pods -o wide | grep csi
```

On EC2 worker:

```
sudo journalctl -u kubelet -n 200
```

Why node logs?

Actual filesystem mount operations happen at the worker level through kubelet/CSI node components.

FIX

Correct the exact mount-layer problem, not StorageClass blindly.

58. FailedMount

OVERVIEW

FailedMount is an Event category, not one specific storage technology.

It can involve:

- Secret;
- ConfigMap;
- PVC;
- CSI;
- EFS;
- filesystem.

BREAK

Reference missing Secret.

DIAGNOSE

```
kubectl describe pod <pod>
```

The full Event matters.

Example:

```
FailedMount:  
secret "x" not found
```

versus:

```
FailedMount:  
mount failed
```

Different causes, same high-level reason.

FIX

Fix the exact volume named in the Event.

COMBINE-OR-SPLIT

Combine with Volume Mount Failure.

59. FailedAttachVolume

OVERVIEW

The disk exists, but Kubernetes/AWS cannot attach it to the intended EC2 node.

ROOT CAUSES

1. AZ mismatch
2. attachment limit
3. IAM
4. KMS
5. stale attachment
6. EC2 node failure

DIAGNOSE

```
kubectl describe pod <pod>
```

Then:

```
kubectl get volumeattachment
```

Describe:

```
kubectl describe volumeattachment <name>
```

AWS:

```
aws ec2 describe-volumes \
  --volume-ids "$VOLUME_ID" \
  --region "$REGION"
```

Why AWS?

Kubernetes reports the desired operation.

EC2 tells you the actual EBS attachment state.

FIX

Correct AZ/scheduling/IAM/stale attachment based on evidence.

60. Multi-Attach Error

BREAK

Use:

```
1 EBS RW0 PVC
+
2 replicas
+
different EC2 nodes
```

Why?

Both pods attempt to use a volume in a pattern that its attachment mode cannot satisfy.

DIAGNOSE

```
kubectl describe pod <second-pod>
```

Look for:

```
Multi-Attach error
```

Then:

```
kubecttl get volumeattachment
```

This identifies the existing node attachment.

FIX

For StatefulSet:

```
volumeClaimTemplates:
```

allows each replica to receive a separate claim.

For shared filesystem, choose architecture such as EFS where appropriate.

61. StatefulSet Volume Problems

OVERVIEW

StatefulSets combine stable pod identity with persistent storage identity.

Typical:

```
app-0 -> data-app-0  
app-1 -> data-app-1
```

ROOT CAUSES

1. PVC Pending
2. StorageClass
3. old PVC retained
4. AZ issue
5. permissions
6. attach issue

DIAGNOSE

```
kubectl get statefulset
kubectl get pods
kubectl get pvc
```

Map pod -> claim:

```
kubectl get pod <pod> \
-o jsonpath='{.spec.volumes[*].persistentVolumeClaim.claimName}'
```

Then investigate that exact claim.

FIX

Repair per-replica storage rather than deleting all StatefulSet data blindly.

62. Pod Moved to Another Node and Volume Won't Attach

OVERVIEW

This occurs during:

- node failure;
- eviction;
- managed-node replacement;
- update.

Old node may still hold the EBS attachment.

DIAGNOSE

New location:

```
kubectl get pod <pod> -o wide
```

Kubernetes attachment:

```
kubectl get volumeattachment
```

AWS:

```
aws ec2 describe-volumes \
  --volume-ids "$VOLUME_ID" \
  --query 'Volumes[0].Attachments'
```

Why?

You want to answer:

Which EC2 instance does AWS currently believe owns the attachment?

FIX

Prefer safe CSI detach/reconciliation.

Do not immediately force-detach a potentially mounted filesystem in production because unsafe simultaneous access can risk data corruption.

63. Disk Full

OVERVIEW

Always determine **which disk** is full:

1. node root filesystem
2. container ephemeral storage
3. PVC
4. inode capacity

DIAGNOSE INSIDE POD

```
kubectl exec <pod> -- df -h
```

Inodes:

```
kubectl exec <pod> -- df -i
```

Node:

```
kubectl describe node <node>
```

Look for:

DiskPressure

On disposable EC2 worker:

```
df -h  
df -i
```

Why `df -i`?

A filesystem can have free bytes but no free inodes.

Then file creation still fails.

FIX

Depending on layer:

- clean/rotate logs;
- manage container images;
- set ephemeral storage requests;
- expand PVC;
- enlarge node root storage;
- replace badly sized node configuration.

64. Permission Denied on Mounted Volume

BREAK

Run app as UID:

```
securityContext:  
  runAsUser: 10001
```

Mount directory owned by root without matching write permission.

DIAGNOSE

```
kubectl exec <pod> -- id
```

Then:

```
kubectl exec <pod> -- ls -ld /data
```

Then test:

```
kubectl exec <pod> -- touch /data/test
```

Why this order?

We establish:

```
process UID/GID
```

then:

```
filesystem ownership/mode
```

then:

```
actual write behavior
```

FIX

Where appropriate:

```
securityContext:  
  fsGroup: 10001
```

or configure backend/access-point ownership correctly.

Avoid teaching:

```
chmod 777
```

as the default solution.

Section 6 - Resources & Scaling

65. CPU Throttling

OVERVIEW

CPU limits can throttle a container even while the EC2 node has spare CPU.

Unlike memory limit violations, CPU overuse doesn't normally kill the container; its CPU time is constrained.

BREAK

```
resources:  
  requests:  
    cpu: 100m  
  limits:  
    cpu: 100m
```

Run a CPU-intensive workload.

DIAGNOSE

Basic:

```
kubectl top pod <pod> --containers
```

For real observability, teach metrics such as:

```
container_cpu_cfs_throttled_seconds_total  
container_cpu_cfs_throttled_periods_total
```

Why `kubectl top` isn't sufficient alone

It shows utilization, but proper throttling diagnosis often requires runtime/Prometheus CPU throttling metrics.

FIX

- adjust CPU limit;
- optimize workload;
- ensure request is realistic;
- measure latency impact.

66. Memory Limit Exceeded

Use OOMKilled lab.

TEACHING DIFFERENCE

```
CPU limit exceeded  
-> throttle  
  
Memory limit exceeded  
-> process may be OOM killed
```

DIAGNOSE

```
kubectl describe pod <pod>  
kubectl top pod <pod> --containers
```

FIX

Right-size + fix leaks.

COMBINE

OOMKilled.

67. Requests Too High

BREAK

Give each tiny nginx:

```
requests:  
  cpu: "2"  
  memory: 4Gi
```

even though it needs a fraction.

DIAGNOSE

Node promises:

```
kubectl describe node <node>
```

Find:

```
Allocated resources  
Requests
```

Actual usage:

```
kubectl top node  
kubectl top pod
```

Teaching comparison

```
Requested 2 CPU  
Actual usage 0.05 CPU
```

The scheduler still reserves 2 CPU.

FIX

Use measured workload behavior and appropriate headroom.

Requests are capacity-planning signals, not a random number copied from another application.

68. Limits Incorrectly Configured

ROOT CAUSES

1. CPU too low
2. memory too low
3. arbitrary request=limit
4. sidecar not accounted
5. workload bursts ignored

DIAGNOSE

```
kubectl get deployment <deployment> \
-o yaml
```

Review:

```
resources.requests
resources.limits
```

Compare:

```
kubectl top pods --containers
```

Then correlate:

- latency;
- throttling;
- OOM;
- scheduling.

COMBINE-OR-SPLIT

Combine with CPU throttling + memory.

69. HPA Not Scaling

OVERVIEW

HPA is a control loop.

For CPU utilization, simplified logic is conceptually:

```
desired replicas ≈
current replicas ×
current utilization / target utilization
```

It also depends on usable metrics and resource definitions.

ROOT CAUSES

1. Metrics Server unavailable
2. request missing
3. load insufficient
4. maxReplicas reached
5. HPA target wrong
6. stabilization/behavior
7. workload not Ready

DIAGNOSE

```
kubectl get hpa
```

Then:

```
kubectl describe hpa <hpa>
```

Why?

describe gives:

- current metric;
- desired metric;
- conditions;
- scaling decisions/errors.

Then:

```
kubectl top pods
```

Metrics API:

```
kubectl get apiservice v1beta1.metrics.k8s.io
```

FIX

Fix metric pipeline/request definitions/threshold.

Verify live:

```
kubectl get hpa,pods -w
```

70. Metrics Server Unavailable

OVERVIEW

Metrics Server backs commands such as:

```
kubectl top
```

and common resource-metric HPAs.

DIAGNOSE

```
kubectl top nodes
```

Failure indicates symptom.

Check:

```
kubectl get apiservice \
  v1beta1.metrics.k8s.io
```

Then find Metrics Server:

```
kubectl get deployment -A | grep metrics-server
```

Logs:

```
kubectl -n kube-system logs deployment/metrics-server
```

if installed there.

FIX

Restore/install compatible Metrics Server and solve kubelet/API connectivity errors.

71. HPA Shows <unknown>

OVERVIEW

<unknown> means HPA cannot obtain or calculate the metric.

DIAGNOSE

```
kubectl describe hpa <hpa>
```

Look for:

```
FailedGetResourceMetric
```

or:

```
FailedComputeMetricsReplicas
```

Then:

```
kubectl top pods
```

Check requests:

```
kubectl get deployment <deployment> -o yaml
```

Why CPU requests matter

For utilization percentage, HPA needs a reference denominator.

Without CPU request, percent utilization may be impossible to calculate correctly for that container/workload.

FIX

Restore metrics and requests.

72. Cluster Autoscaler Doesn't Add Nodes

OVERVIEW

Cluster Autoscaler watches unschedulable pods and simulates whether adding nodes from available node groups would make them schedulable. Its version should match the cluster version and it should use properly scoped AWS permissions and node-group discovery.

ROOT CAUSES

1. no genuinely unschedulable pod
2. maximum node-group size reached
3. impossible affinity

4. IAM denied
5. auto-discovery/tag configuration
6. version/configuration
7. EC2 capacity issue

BREAK

Create CPU request too large for existing capacity but feasible on an additional node.

DIAGNOSE

Pod:

```
kubectl describe pod <pod>
```

Autoscaler:

```
kubectl -n kube-system logs \
  deployment/cluster-autoscaler \
  --tail=300
```

Why logs?

Autoscaler explains whether it considered the pod and why a scale-up was or wasn't triggered.

Node-group limits:

```
aws eks describe-nodegroup \
  --cluster-name "$CLUSTER" \
  --nodegroup-name "$NG" \
  --region "$REGION" \
  --query nodegroup.scalingConfig
```

If:

```
desiredSize = maxSize
```

the group cannot expand further.

FIX

Evidence-specific:

- increase max;
- correct IAM;
- fix auto-discovery;
- fix impossible constraints;

- resolve EC2 capacity.

COMBINE-OR-SPLIT

Standalone flagship.

73. Pod Can't Schedule After HPA Scaling

OVERVIEW

HPA scales **pods**.

Cluster Autoscaler/Karpenter/EKS compute scaling handles **nodes**.

These are separate control loops.

BREAK

Create load causing HPA to increase replicas while workers have insufficient remaining capacity.

DIAGNOSE

```
kubectl get hpa,pods
```

You'll see:

```
desired replicas increased  
but  
some replicas Pending
```

Then:

```
kubectl describe pod <pending-pod>
```

The scheduler still gives the root cause.

FIX

Restore node-scaling path or correct workload placement/resource configuration.

Section 7 - Configuration

74. ConfigMap Not Loaded

ROOT CAUSES

1. wrong ConfigMap name
2. wrong key
3. wrong namespace
4. environment value not refreshed
5. mounted app doesn't reload
6. `subPath` behavior

BREAK

Create:

```
kubectl create configmap app-config \
  --from-literal=MODE=development
```

Inject it as env.

Then change:

```
kubectl patch configmap app-config \
  --type merge \
  -p '{"data":{"MODE":"production"}}'
```

Why can pod still show old value?

Environment variables are set when the container starts.

Updating ConfigMap doesn't rewrite a running process's environment.

DIAGNOSE

```
kubectl get configmap app-config -o yaml
```

shows new value.

But:

```
kubectl exec <pod> -- printenv MODE
```

may show old value.

This is the teaching proof.

FIX

For env-based config:

```
kubectl rollout restart deployment/<deployment>
```

New pods read updated environment.

75. Secret Not Available

ROOT CAUSES

1. wrong name
2. wrong namespace
3. wrong key
4. secret missing
5. stale env
6. external synchronization failure

DIAGNOSE

```
kubectl get secret
```

Then:

```
kubectl describe pod <pod>
```

If environment-based:

```
kubectl get deployment <deployment> -o yaml
```

Check:

```
secretKeyRef
```

Safe lab decode

```
kubectl get secret <secret> \
  -o jsonpath='{.data.username}' | base64 -d
echo
```

Explain that base64 is encoding, not encryption itself.

FIX

Correct object/key and restart workloads if env-based injection needs recreation.

76. Environment Variable Missing

BREAK

Application expects:

```
API_URL
```

manifest defines:

```
API_URL
```

lowercase `l`.

DIAGNOSE

```
kubectl exec <pod> -- env | sort
```

Why?

This shows the environment the process actually received.

Then:

```
kubectl get deployment <deployment> -o yaml
```

Compare desired vs actual.

FIX

Correct exact variable name and rollout.

77. Incorrect YAML Indentation

OVERVIEW

Two failure classes:

```
Invalid YAML syntax
```

and:

Valid YAML, wrong Kubernetes structure

DIAGNOSE BEFORE APPLY

```
kubectl apply \  
  --dry-run=client \  
  -f manifest.yaml
```

Then API schema:

```
kubectl apply \  
  --dry-run=server \  
  -f manifest.yaml
```

Why both?

Client dry-run can catch parsing/basic issues.

Server dry-run validates against the cluster API without persisting the object.

FIX

Correct structure and use:

```
kubectl explain deployment.spec.template.spec
```

to learn expected schema.

78. Invalid API Version

BREAK

```
apiVersion: extensions/v1beta1  
kind: Deployment
```

on a modern cluster.

DIAGNOSE

```
kubectl apply -f old.yaml
```

Then:

```
kubectl api-versions  
kubectl api-resources
```

Why?

Don't guess supported APIs.

Ask the actual EKS Kubernetes API.

FIX

For Deployment:

```
apiVersion: apps/v1
```

and update associated required fields.

COMBINE-OR-SPLIT

Excellent EKS-upgrade companion.

79. Strict Decoding Error

BREAK

```
resources:
  limits:
    memory: 256Mi
```

DIAGNOSE

```
kubectl apply \
  --dry-run=server \
  -f broken.yaml
```

Then:

```
kubectl explain \
  deployment.spec.template.spec.containers.resources
```

The API server is essentially saying:

I know this object schema, but `limits` isn't a valid field here.

FIX

```
limits:
```

80. Typo Such as `llimits`

Use this as the smallest version of strict-decoding training.

CLASSROOM EXERCISE

Give students five manifests containing:

```
limits
request
container
readinessProbe
matchLabels
```

Have them diagnose using:

```
kubectl apply --dry-run=server
kubectl explain
```

COMBINE-OR-SPLIT

Combine with strict decoding.

81. Deployment Selector Mismatch

BREAK

```
selector:
  matchLabels:
    app: web
template:
  metadata:
    labels:
      app: api
```

Why invalid?

The Deployment claims to manage pods selected by:

```
app=web
```

but creates:

```
app=api
```

The controller could not consistently manage its own pod set.

DIAGNOSE

```
kubectl apply -f deployment.yaml
```

API validation should reject the mismatch.

Use:

```
kubectl get deployment <deployment> -o yaml
```

for existing resources.

FIX

Make selector and template labels match.

82. Wrong Namespace

BREAK

Create:

```
kubectl create secret generic database \
  --from-literal=password=demo \
  -n default
```

Pod in:

```
breakfix
```

references Secret `database`.

Why?

Secrets are namespace-scoped.

The pod cannot access a Secret of the same name in another namespace.

DIAGNOSE

```
kubectl get secret database -n breakfix
```

Not found.

Then:

```
kubectl get secret database -n default
```

Found.

Root cause proven.

FIX

Create/reference the resource in the correct namespace.

Section 8 - RBAC and Security

83. Forbidden Error

OVERVIEW

Teach students three different categories:

Unauthenticated

-> Who are you?

Forbidden

-> I know who you are, but you're not allowed.

NotFound

-> Authorized request but resource absent/hidden as applicable

For IAM users/roles, EKS Access Entries are now the recommended mechanism; the legacy `aws-auth` ConfigMap is deprecated for this use.

DIAGNOSE

AWS identity:

```
aws sts get-caller-identity
```

Why?

It tells us which IAM identity obtains the EKS authentication token.

Kubernetes identity where supported:

```
kubectl auth whoami
```

Authorization:

```
kubectl auth can-i get pods -n "$NS"
```

And:

```
kubectl auth can-i --list -n "$NS"
```

EKS access:

```
aws eks list-access-entries \  
  --cluster-name "$CLUSTER" \  
  --region "$REGION"
```

FIX

Use appropriate access entry / Kubernetes RBAC.

Do not grant cluster-admin simply to make an error disappear.

COMBINE-OR-SPLIT

Standalone flagship.

84. ServiceAccount Permission Denied

OVERVIEW

Two independent security systems may be involved:

```
Kubernetes API authorization -> RBAC  
AWS API authorization -> IAM
```

BREAK - Kubernetes RBAC

Create ServiceAccount:

```
kubectl create serviceaccount student-app
```

Check:

```
kubectl auth can-i list secrets \  
  --as=system:serviceaccount:"$NS":student-app \  
  -n "$NS"
```

Expected:

```
no
```

Why?

ServiceAccount authentication exists but it hasn't been granted that permission.

FIX RBAC

Example safe read role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: breakfix
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Binding:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: breakfix
subjects:
- kind: ServiceAccount
  name: student-app
  namespace: breakfix
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-reader
```

Verify:

```
kubectl auth can-i list pods \
  --as=system:serviceaccount:"$NS":student-app \
  -n "$NS"
```

AWS IAM CASE

Pod Identity trust relationship:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "pods.eks.amazonaws.com"
      },
      "Action": [
        "sts:AssumeRole",
        "sts:TagSession"
      ]
    }
  ]
}
```

Example minimal lab permission:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::YOUR-LAB-BUCKET"
    }
  ]
}
```

Associate:

```
aws eks create-pod-identity-association \
  --cluster-name "$CLUSTER" \
  --namespace "$NS" \
  --service-account student-app \
  --role-arn "arn:aws:iam::$ACCOUNT_ID:role/StudentAppRole" \
  --region "$REGION"
```

AWS recommends Pod Identity for these workload/add-on IAM associations while IRSA remains supported.

TEACHING POINT

If:

```
kubectl get pods
```

fails:

Think RBAC.

If:

```
aws s3 ls
```

inside the pod returns AccessDenied:
Think IAM.

85. Role vs ClusterRole

OVERVIEW

Role:

```
namespace-scoped permissions
```

ClusterRole:

```
can define cluster-scoped permissions  
or reusable permissions bindable into namespaces
```

BREAK

Give a namespace Role permission to pods, then attempt:

```
kubectl auth can-i list nodes \  
--as=system:serviceaccount:"$NS":student-app
```

Nodes are cluster-scoped.

DIAGNOSE

```
kubectl api-resources \  
--namespaced=true
```

and:

```
kubectl api-resources \  
--namespaced=false
```

Why?

This teaches students whether a resource belongs to a namespace at all.

FIX

Use minimal ClusterRole where cluster-scope is genuinely needed.

86. RoleBinding Misconfiguration

ROOT CAUSES

1. wrong subject name
2. wrong subject namespace
3. wrong roleRef
4. RoleBinding in wrong namespace
5. ServiceAccount mismatch

DIAGNOSE

```
kubectl get rolebinding <binding> -o yaml
```

Then test directly:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:"$NS":student-app \
  -n "$NS"
```

Why is this command excellent?

It allows you to test authorization without repeatedly deploying a pod.

FIX

Correct exact subject and roleRef.

87. Image Registry Authentication

See ImagePullBackOff, but emphasize identity layers.

For EC2-backed EKS private ECR pulls, inspect the worker/node permissions used by the image-pull path.

For third-party registries, inspect ImagePullSecret.

DIAGNOSE

```
kubectl describe pod <pod>
```

Interpret:

```
unauthorized
denied
manifest unknown
timeout
x509
```

Each suggests a different layer.

COMBINE

ImagePullBackOff.

88. ImagePullSecret Problems

BREAK

Create secret in:

```
default
```

but pod runs in:

```
breakfix
```

DIAGNOSE

```
kubectl get secret regcred -n breakfix
```

Missing.

```
kubectl get secret regcred -n default
```

Exists.

FIX

Create Secret in workload namespace and reference:

```
imagePullSecrets:
- name: regcred
```

89. Certificate Expired

OVERVIEW

On EKS, separate:

from certificates you operate:

- Ingress;
- webhook;
- internal mTLS;
- custom trust;
- clients.

DIAGNOSE EXTERNAL TLS

```
openssl s_client \
  -connect app.example.com:443 \
  -servername app.example.com \
  </dev/null 2>/dev/null |
openssl x509 \
  -noout \
  -dates \
  -subject \
  -issuer
```

What do these fields mean?

```
notBefore -> certificate valid from
notAfter  -> expiration
subject   -> identity
issuer    -> signing CA
```

FIX

Renew/replace the certificate in the owning system, such as ACM/ internal PKI, rather than attempting kubeadm control-plane certificate renewal on EKS.

90. TLS Certificate Error

ROOT CAUSES

1. expired
2. untrusted CA
3. hostname/SAN mismatch
4. missing chain
5. SNI
6. wrong ALB certificate
7. webhook CA mismatch

DIAGNOSE

```
curl -vk https://app.example.com
```

Then:

```
openssl s_client \  
-connect app.example.com:443 \  
-servername app.example.com \  
-showcerts </dev/null
```

Why both?

`curl` shows client/application failure.

`openssl` exposes detailed certificate negotiation and chain.

FIX

Correct the actual trust/identity problem.

Never present:

```
curl -k
```

as the fix.

It disables validation and is only a diagnostic shortcut.

Section 9 - Node and Control Plane

91. Node NotReady

OVERVIEW

The Kubernetes control plane has lost healthy communication/status from kubelet or the node has reported conditions preventing readiness.

ROOT CAUSES

1. kubelet stopped
2. API connectivity
3. CNI/networking
4. OS problem
5. resource pressure
6. instance failure

7. node bootstrap/identity

BREAK LAB

On a disposable EC2 worker:

```
sudo systemctl stop kubelet
```

What does it do?

kubelet is the node agent responsible for:

- node status;
- pod lifecycle;
- communication with the API server.

Once stopped, the control plane eventually stops receiving node heartbeats.

DIAGNOSE

```
kubectl get nodes
```

Watch transition.

Then:

```
kubectl describe node <node>
```

Conditions:

```
kubectl get node <node> \
-o jsonpath='{range .status.conditions[*]}{.type}{"="}{.status}{" reason="}
  {.reason}{"\n"}{end}'
```

On node:

```
sudo systemctl status kubelet
```

Why?

Directly verifies service state.

Logs:

```
sudo journalctl -u kubelet -n 200
```

Why?

A stopped/failed service tells you **that kubelet isn't healthy**.

Logs tell you **why**.

FIX

For our deliberate lab:

```
sudo systemctl start kubelet
```

Verify:

```
kubectl get nodes -w
```

Expected:

```
NotReady -> Ready
```

COMBINE-OR-SPLIT

Standalone flagship.

92. kubelet Not Running

Use the previous stop/start lab.

ADDITIONAL DIAGNOSIS

```
sudo systemctl status kubelet  
sudo journalctl -xeu kubelet
```

Runtime:

```
sudo systemctl status containerd
```

Disk:

```
df -h
```

Ports:

```
sudo ss -lntp | grep 10250
```

Teaching reasoning

A kubelet service may fail because:

```
kubelet itself  
OR  
dependency/config/OS problem
```

So don't blindly run `systemctl start` repeatedly without reading logs.

93. kubelet node not found

OVERVIEW

Kubelet may be running but the Node API object doesn't exist or registration/authentication is failing.

BREAK LAB

On a disposable node:

```
kubectrl delete node <node-name>
```

Important explanation

This deletes the **Kubernetes Node object**.

It does not terminate the underlying EC2 instance.

If kubelet remains correctly configured and authorized, it may register again.

DIAGNOSE

On node:

```
sudo journalctl -u kubelet -f
```

Look for:

```
node not found  
registration  
unauthorized  
forbidden
```

Inspect API endpoint in kubelet config as appropriate.

FIX

If this is merely the lab deletion, healthy kubelet registration should restore the object.

If not, investigate:

- IAM/node access;

- bootstrap;
- cluster endpoint;
- kubelet configuration.

94. DiskPressure

OVERVIEW

This is a **node condition**, not necessarily an application PVC problem. Kubelet monitors node filesystem availability.

SAFE BREAK LAB

Only on a disposable worker.

First:

```
df -h
```

Do not blindly fill 100% of the disk.

Create a bounded file appropriate to your lab disk:

```
sudo fallocate -l 2G /var/tmp/breakfix-disk-file
```

Recheck:

```
df -h
```

Increase only if your disposable environment safely permits it.

DIAGNOSE

```
kubectl describe node <node>
```

Look for:

```
DiskPressure
```

On node:

```
df -h
df -i
sudo du -xhd1 /var/log /var/lib/containerd 2>/dev/null
```

Why these?

`df -h:`

bytes.

`df -i:`

inodes.

`du:`

what directories consume space.

FIX

Remove lab file:

```
sudo rm -f /var/tmp/breakfix-disk-file
```

Production:

- log rotation;
- image GC;
- correct node volume sizing;
- ephemeral storage design.

95. MemoryPressure

OVERVIEW

Node-wide memory scarcity differs from a single container exceeding its limit.

DIAGNOSE

```
kubectl describe node <node>
```

Then:

```
kubectl top node <node>
```

Pod ranking:

```
kubectl top pods -A \
--sort-by=memory
```

On node:

```
free -m
```

Kernel evidence:

```
sudo dmesg | grep -i -E 'oom|out of memory'
```

Why?

The Kubernetes condition tells us node state.

`top` identifies consumers.

Kernel logs may reveal actual OOM decisions.

FIX

- reduce memory consumption;
- right-size requests/limits;
- move workload;
- larger/more nodes;
- fix leaks.

96. PIDPressure

OVERVIEW

Nodes have finite process IDs.

A runaway application creating huge numbers of processes/threads can exhaust that resource.

SAFE BREAK LAB

Do not teach an unrestricted fork bomb.

Use a bounded loop in a disposable pod/node:

```
for i in $(seq 1 100); do
  sleep 300 &
done
```

This creates a controlled number of processes.

Cleanup:

```
pkill sleep
```

DIAGNOSE

```
kubectl describe node <node>
```

Look for PIDPressure when threshold is genuinely approached.

Node:

```
ps -eLf | wc -l
```

Maximum:

```
cat /proc/sys/kernel/pid_max
```

Teaching point

Threads may consume task/PID resources too, so a “process explosion” can actually come from a heavily threaded application.

97. API Server Unavailable

OVERVIEW

EKS control plane is AWS managed.

Therefore don't teach students:

```
SSH master  
systemctl restart kube-apiserver
```

Instead investigate:

- cluster state;
- API endpoint;
- private/public access;
- network route;
- client config;
- AWS control-plane logs.

EKS can publish API, audit, authenticator, scheduler and controller-manager logs to CloudWatch.

DIAGNOSE

```
aws eks describe-cluster \
  --name "$CLUSTER" \
  --region "$REGION" \
  --query 'cluster.{Status:status,Endpoint:endpoint,Network:resourcesVpcConfig}'
```

Why?

This asks AWS directly rather than relying on Kubernetes API connectivity.

Extract endpoint:

```
ENDPOINT=$(aws eks describe-cluster \
  --name "$CLUSTER" \
  --region "$REGION" \
  --query cluster.endpoint \
  --output text)
```

Test TLS reachability:

```
curl -vk "$ENDPOINT"
```

Interpretation

Timeout:

network path / endpoint accessibility

HTTP/TLS response:

network path exists; authentication is a separate layer

FIX

Correct endpoint access/VPC route/VPN/bastion/corporate firewall/kubeconfig depending on evidence.

98. kubectl Connection Refused

OVERVIEW

Connection refused means TCP connection was rejected.

This is different from:

```
Unauthorized
Forbidden
certificate error
timeout
```

DIAGNOSE

```
kubectl config current-context
```

Then:

```
kubectl config view --minify
```

Inspect:

```
server:
```

Compare with:

```
aws eks describe-cluster \
  --name "$CLUSTER" \
  --region "$REGION" \
  --query cluster.endpoint \
  --output text
```

FIX

Refresh kubeconfig:

```
aws eks update-kubeconfig \
  --name "$CLUSTER" \
  --region "$REGION"
```

Verify:

```
kubectl get nodes
```

99. localhost:8080 refused

OVERVIEW

This classic symptom often indicates kubectl isn't using a valid cluster configuration and is attempting a local endpoint.

It is not evidence that “EKS API server is down.”

BREAK LAB

Use a shell with a deliberately empty kubeconfig:

```
export KUBECONFIG=/tmp/empty-kubeconfig  
touch "$KUBECONFIG"
```

Run:

```
kubectl get nodes
```

Depending on kubectl version/config behavior, students can observe missing/invalid configuration errors and learn why localhost-style errors occur.

DIAGNOSE

```
echo "$KUBECONFIG"  
kubectl config get-contexts  
kubectl config view
```

Why?

The entire problem may exist on the client before any packet ever reaches AWS.

FIX

```
unset KUBECONFIG  
  
aws eks update-kubeconfig \  
  --name "$CLUSTER" \  
  --region "$REGION"
```

Then:

```
kubectl cluster-info
```

100. etcd Unavailable - EKS Adaptation

OVERVIEW

This is where the course must clearly differ from kubeadm courses.

In EKS:

```
Control-plane etcd  
=  
AWS-managed
```

Students do not:

```
ssh master  
systemctl restart etcd  
etcdctl member list  
edit /etc/kubernetes/manifests/etcd.yaml
```

on the managed EKS control plane.

WHAT SHOULD STUDENTS TROUBLESHOOT INSTEAD?

Symptoms that generic courses may attribute to etcd:

1. API unavailable
2. API latency
3. AWS EKS control-plane health issue
4. client cannot reach private endpoint
5. authentication failure
6. admission/webhook latency

DIAGNOSE

AWS:

```
aws eks describe-cluster \  
  --name "$CLUSTER" \  
  --region "$REGION"
```

API:

```
kubectl get --raw='/readyz?verbose'
```

when API remains reachable and authorization allows it.

Enable control-plane logging:

```
aws eks update-cluster-config \
  --name "$CLUSTER" \
  --region "$REGION" \
  --logging '{
    "clusterLogging":[
      {
        "types":[
          "api",
          "audit",
          "authenticator",
          "controllerManager",
          "scheduler"
        ],
        "enabled":true
      }
    ]
  }'
```

AWS supports these EKS control-plane log categories through CloudWatch rather than direct customer host access.

FIX

For an actual AWS-managed control-plane failure, customers investigate EKS/AWS health and engage AWS support when necessary rather than attempting direct etcd remediation.

VIDEO

etcd Is Down? Why kubeadm etcd Troubleshooting Does Not Apply to Amazon EKS

101. etcd TLS Handshake Failure - EKS Adaptation

OVERVIEW

Again, do not fake direct access to EKS etcd.

Use this topic to teach the difference between:

self-managed etcd TLS

and EKS TLS errors customers can actually own:

- kubeconfig trust;
- admission webhooks;
- application mTLS;
- Ingress TLS.

DIAGNOSE CLIENT TRUST

```
kubectl config view --raw --minify
```

Refresh:

```
aws eks update-kubeconfig \  
  --name "$CLUSTER" \  
  --region "$REGION"
```

Webhook:

```
kubectl get validatingwebhookconfiguration -o yaml
```

Look at:

```
clientConfig.caBundle
```

COMBINE-OR-SPLIT

Combine with etcd unavailable in **EKS Managed Control Plane vs kubeadm Troubleshooting**.

102. etcd Port 2379 Already in Use - EKS Adaptation

OVERVIEW

Port 2379 is a common self-managed etcd client port.

On managed EKS, students shouldn't be logging into AWS-owned control-plane nodes and freeing that port.

CLASSROOM LESSON

Show the generic error:

```
bind: address already in use :2379
```

Then explain:

In kubeadm/self-managed Kubernetes:

```
investigate control-plane host
```

In EKS:

```
you do not administer that control-plane host
```

Use:

```
aws eks describe-cluster \
  --name "$CLUSTER" \
  --region "$REGION"
```

and managed diagnostics instead.

COMBINE-OR-SPLIT

Combine with other EKS control-plane boundary topics.

103. Port 10250 Already in Use

OVERVIEW

This one **can** be relevant on EC2 EKS workers because kubelet uses its HTTPS serving port.

BREAK LAB - DISPOSABLE WORKER ONLY

Stop kubelet:

```
sudo systemctl stop kubelet
```

Check port:

```
sudo ss -lntp | grep 10250
```

Start a temporary listener:

```
sudo python3 -m http.server 10250 \
  --bind 0.0.0.0 &
```

Restart kubelet:

```
sudo systemctl start kubelet
```

DIAGNOSE

```
sudo systemctl status kubelet
```

Then:

```
sudo journalctl -u kubelet -n 100
```

Look for:

```
address already in use
```

Confirm owner:

```
sudo ss -lntp | grep 10250
```

Why `ss`?

It identifies:

```
port  
+  
listening process
```

instead of guessing.

FIX

Terminate our lab process:

```
sudo pkill -f 'http.server 10250'
```

Then:

```
sudo systemctl restart kubelet
```

Verify:

```
sudo ss -lntp | grep 10250  
kubectl get node <node>
```

104. CoreDNS Not Ready

OVERVIEW

A CoreDNS container may exist but fail readiness.

DNS capacity then becomes degraded.

DIAGNOSE

```
kubectl -n kube-system get pods \\\n  -l k8s-app=kube-dns
```

Then:

```
kubectl -n kube-system describe pod <coredns-pod>
```

Logs:

```
kubectl -n kube-system logs <coredns-pod>
```

Configuration:

```
kubectl -n kube-system get cm coredns -o yaml
```

EKS add-on:

```
aws eks describe-addon \  
  --cluster-name "$CLUSTER" \  
  --addon-name coredns \  
  --region "$REGION"
```

FIX

Correct scheduler/resources/configuration/add-on health depending on evidence.

105. kube-proxy Failure

Use the networking kube-proxy lab.

ADDITIONAL TEACHING

Show three tests:

```
Pod -> Pod IP  
Pod -> Service ClusterIP  
Pod -> DNS Service
```

If:

```
Pod IP succeeds  
Service IP fails
```

students should narrow focus toward Service dataplane.

Check kube-proxy per worker:

```
kubectl -n kube-system get pods \  
  -l k8s-app=kube-proxy \  
  -o wide
```

106. CNI Not Initialized

OVERVIEW

A worker can register with the API server before pod networking is ready.

DIAGNOSE

```
kubectl describe node <node>
```

Look for:

```
NetworkPluginNotReady
```

or:

```
cni plugin not initialized
```

Then:

```
kubectl -n kube-system get pods \
  -l k8s-app=aws-node \
  -o wide
```

Logs:

```
kubectl -n kube-system logs \
  ds/aws-node \
  -c aws-node \
  --tail=200
```

Why?

This directly inspects the EKS VPC networking agent responsible for the EC2 worker's pod networking path.

FIX

Correct:

- CNI DaemonSet;
- IAM;
- add-on configuration;
- ENI/subnet capacity;
- node bootstrap.

107. Certificate Problems

OVERVIEW

Split certificate troubleshooting into ownership domains:

```
kubectl client
kubelet worker
webhook
Ingress
application TLS
AWS-managed EKS control plane
```

Never mix them.

DIAGNOSE KUBELET

On EC2 worker:

```
sudo journalctl -u kubelet |
grep -iE 'x509|certificate|tls'
```

System clock:

```
timedatectl
date -u
```

Why time?

Certificate validity depends on time.

Large clock skew can make a valid certificate appear:

```
not yet valid
```

or:

```
expired
```

DIAGNOSE kubectl

```
kubectl config view --raw --minify
```

Refresh:

```
aws eks update-kubeconfig \
  --name "$CLUSTER" \
  --region "$REGION"
```

FIX

Repair the certificate/trust relationship owned by that layer.

Do **not** run:

```
kubeadm certs renew all
```

against AWS-managed EKS masters.

108. kubeadm Upgrade Problems - EKS Managed Upgrade Version

OVERVIEW

This topic must be completely rewritten for EKS.

With Amazon EKS:

```
AWS manages control-plane nodes
```

You don't execute:

```
kubeadm upgrade apply
```

on an EKS control plane.

Instead the upgrade lifecycle involves:

```
Application/API compatibility
↓
EKS control-plane version
↓
Add-ons
↓
Managed node groups
↓
Workload validation
```

Managed node-group updates are performed through EKS; normal node-group updates create/prepare replacement capacity, cordon/drain old nodes and can fail with `PodEvictionFailure` if pods can't be evicted.

ROOT CAUSE 1 - Deprecated API

BREAK

Keep an old lab manifest:

```
apiVersion: extensions/v1beta1
kind: Deployment
```

WHY THIS REPRESENTS A REAL UPGRADE PROBLEM

An application may have worked on an older Kubernetes API version but stop applying when that API is removed.

DIAGNOSE

```
kubectl api-versions
kubectl api-resources
```

Test manifests against current API:

```
kubectl apply \
  --dry-run=server \
  -f manifests/
```

FIX

Migrate manifests/operators/Helm charts before upgrading the cluster.

ROOT CAUSE 2 - EKS Add-on Compatibility

DIAGNOSE

List:

```
aws eks list-addons \
  --cluster-name "$CLUSTER" \
  --region "$REGION"
```

Inspect:

```
aws eks describe-addon \
  --cluster-name "$CLUSTER" \
  --addon-name vpc-cni \
  --region "$REGION"
```

Discover versions valid for the target Kubernetes version:

```
aws eks describe-addon-versions \
  --addon-name vpc-cni \
  --kubernetes-version <TARGET_VERSION> \
  --region "$REGION"
```

Repeat for:

```
coredns  
kube-proxy  
aws-ebs-csi-driver
```

WHY?

Control-plane upgrade and cluster add-on compatibility are related but separate lifecycle operations.

ROOT CAUSE 3 - Managed Node Group Behind Control Plane

DIAGNOSE

Control plane:

```
aws eks describe-cluster \  
  --name "$CLUSTER" \  
  --region "$REGION" \  
  --query cluster.version
```

Node group:

```
aws eks describe-nodegroup \  
  --cluster-name "$CLUSTER" \  
  --nodegroup-name "$NG" \  
  --region "$REGION" \  
  --query nodegroup.version
```

WHY TWO COMMANDS?

Students can visually compare:

```
Control Plane = X  
Workers = Y
```

FIX

Update managed node group:

```
eksctl upgrade nodegroup \  
  --name "$NG" \  
  --cluster "$CLUSTER" \  
  --region "$REGION"
```

Managed-node-group updating is an EKS operation rather than a user-run kubeadm operation.

ROOT CAUSE 4 - PDB Blocks Drain

BREAK

Single-replica workload with:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: strict-pdb
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: singleton
```

WHAT DOES THIS MEAN?

There is only:

```
1 pod
```

PDB requires:

```
at least 1 remain available
```

Therefore voluntary eviction may be impossible.

DIAGNOSE

```
kubectl get pdb
```

Then:

```
kubectl describe pdb strict-pdb
```

Check:

```
Allowed disruptions
```

If:

```
0
```

drain cannot voluntarily evict the pod under normal conditions.

During managed node-group update, EKS can surface

`PodEvictionFailure` when pods can't be drained within the update workflow.

FIX

For the lab:

- increase replicas;
- provide capacity;
- adjust PDB safely.

Don't teach “force update” as the default.

Force bypasses disruption safeguards and may create availability impact.

ROOT CAUSE 5 - New Node Doesn't Join

DIAGNOSE

Managed node-group health:

```
aws eks describe-nodegroup \
  --cluster-name "$CLUSTER" \
  --nodegroup-name "$NG" \
  --region "$REGION" \
  --query nodegroup.health
```

EC2:

```
aws ec2 describe-instances \
  --filters "Name=tag:eks:nodegroup-name,Values=$NG" \
  --region "$REGION"
```

Worker logs where accessible:

```
sudo journalctl -u kubelet -n 200
```

Check:

- IAM node role;
- security groups;
- API endpoint reachability;
- bootstrap;
- custom AMI;
- launch template.

ROOT CAUSE 6 - Insufficient Temporary Capacity

EKS's default managed-node update behavior can create replacement capacity before old nodes are removed, whereas update strategies affect how capacity is handled.

DIAGNOSE

```
aws autoscaling describe-scaling-activities \
  --region "$REGION"
```

Look for:

- insufficient capacity;
- quota;
- launch failures.

VIDEO STRUCTURE

Hook:

“If your EKS upgrade guide starts with `kubeadm upgrade apply`, stop. That's the wrong control-plane architecture.”

Title:

EKS Upgrade Troubleshooting - APIs, Add-ons, Node Groups, PDBs and Failed Drains

Chapters:

```
00:00 Why kubeadm Doesn't Apply
04:00 API Compatibility
09:00 Add-ons
14:00 Managed Node Groups
20:00 PodDisruptionBudgets
26:00 Node Join Failures
32:00 Capacity Problems
```

COMMON MISTAKES

Never teach these as EKS managed-control-plane fixes:

```
systemctl restart kube-apiserver
systemctl restart etcd
kubeadm upgrade apply
kubeadm certs renew all
```

Those belong to architectures where you administer the control plane.

COMBINE-OR-SPLIT

Standalone flagship EKS video.

It is one of the best ways to differentiate an EKS-focused course from a generic Kubernetes course.

Recommended Classroom Troubleshooting Pattern for Every Demo

Students should eventually memorize this approach.

Step 1 - Identify the object

```
kubectl get pod  
kubectl get deployment  
kubectl get svc  
kubectl get pvc  
kubectl get node
```

Don't troubleshoot an object before knowing which object is unhealthy.

Step 2 - Get detailed state

```
kubectl describe <resource> <name>
```

Ask:

```
What condition failed?  
What Event occurred?  
Which component reported it?
```

Step 3 - Check Events

```
kubectl get events \\\n  --sort-by='.lastTimestamp'
```

Use namespace-specific Events where possible.

Step 4 - Check application/container evidence

```
kubectl logs <pod>  
kubectl logs <pod> --previous  
kubectl logs <pod> -c <container>
```

Students should understand exactly when each is useful.

Step 5 - Check the live runtime

```
kubectl exec <pod> -- env
kubectl exec <pod> -- ss -lnt
kubectl exec <pod> -- cat /etc/resolv.conf
kubectl exec <pod> -- df -h
```

The manifest tells us what we **intended**.

The running container tells us what actually happened.

Step 6 - Check cluster component

Depending on symptom:

```
Scheduling
-> scheduler Events

DNS
-> CoreDNS

Service connectivity
-> Service/EndpointSlice/kube-proxy

Pod networking
-> VPC CNI/aws-node

Persistent volume
-> EBS CSI

Node
-> kubelet/containerd

Autoscaling
-> HPA/Metrics Server/Cluster Autoscaler

Ingress/LoadBalancer
-> AWS Load Balancer Controller + ELB
```

Step 7 - Check AWS

Examples:

```
aws eks describe-cluster
aws eks describe-nodegroup
aws eks describe-addon
aws ec2 describe-subnets
aws ec2 describe-volumes
aws elbv2 describe-target-health
aws iam get-role
aws eks list-access-entries
```

Ask:

Is Kubernetes asking AWS to do something AWS is refusing or unable to do?

Step 8 - Change One Thing

This is extremely important for students.

Bad troubleshooting:

```
change security group
restart pod
change probe
change IAM
restart CNI
scale nodes
```

all at once.

Then if the problem disappears, you don't know why.

Professional troubleshooting:

```
Hypothesis
-> One change
-> Retest
-> Record evidence
```

Step 9 - Verify the Fix

A successful command is not sufficient.

Example:

```
kubectl apply -f fixed.yaml
```

may return:

```
configured
```

That means only:

API server accepted the configuration.

It does not mean:

Application is healthy.

Verify:

```
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
curl <application>
```

and where relevant:

```
kubectl get endpointslice
kubectl get hpa
kubectl get pvc
kubectl get nodes
```

How I Would Teach These Topics to Students

Every lesson should use the same story.

Phase 1 - Healthy Environment

Show students that the workload works first.

Example:

```
kubectl get pods
curl http://application
```

Without a healthy baseline, students don't know whether the later failure was caused by your change.

Phase 2 - Explain the Architecture

Before breaking anything, draw:

```
Client
↓
ALB
↓
Ingress
↓
Service
↓
EndpointSlice
↓
Pod
```

OR:

```
PVC
↓
StorageClass
↓
EBS CSI Controller
↓
AWS EC2 API
↓
EBS Volume
↓
CSI Node
↓
Pod
```

OR:

```
Pod
↓
Scheduler
↓
Node constraints
↓
EC2 worker
```

Students troubleshoot much faster once they understand which component owns each operation.

Phase 3 - Break Exactly One Layer

Tell students before running the command:

```
We are about to deliberately break X.

We expect Y symptom.

Component Z should report the failure.
```

Then execute it.

Phase 4 - Observe Without Fixing

Ask students:

```
What changed?
What did not change?
```

Use:

```
kubectl get
```

first.

Phase 5 - Collect Evidence

Don't tell them the answer immediately.

Run:

```
kubectl describe  
kubectl logs  
kubectl get events
```

and teach them how to derive the conclusion.

Phase 6 - Form the Root-Cause Statement

Students should be able to say something specific like:

"The pod isn't Pending because EKS is down. The scheduler rejected all three workers because the pod requests 8 CPUs while no node has 8 CPUs of unallocated requested capacity."

That is much better than:

"CPU problem."

Phase 7 - Fix

Explain exactly what configuration change removes the constraint.

Phase 8 - Verify at Multiple Layers

For example:

```
Pod Running  
+  
Pod Ready  
+  
Service endpoint present  
+  
HTTP request succeeds
```

Only then close the incident.

Recommended Video Grouping

For the course/channel, I would not turn every line above into an isolated 20-minute video.

Use roughly these major practical modules:

1. **EKS Pod Pending Masterclass**
 - Pending
 - 0/x nodes
 - CPU
 - memory
2. **CrashLoopBackOff Masterclass**
 - CrashLoop
 - exit 1
 - restart loops
3. **OOMKilled & Exit 137**
4. **EKS Image Pull Troubleshooting**
 - ErrImagePull
 - ImagePullBackOff
 - ECR
 - ImagePullSecrets
5. **Container Startup Problems**
6. **ContainerCreating**
 - Secret
 - CNI
 - EBS
7. **Terminating Pods & Finalizers**
8. **Init and Sidecar Failures**
9. **Works Locally but Fails on EKS**
10. **NodeSelector + Node Affinity**
11. **Taints and Tolerations**
12. **Anti-Affinity + Topology Spread**
13. **EKS Pod Limits / ENI / IP Capacity**
14. **Liveness Probes**
15. **Readiness Probes**
16. **Startup Probes**
17. **Probe HTTP/Port/Timeout Errors**
18. **Pod-to-Pod Networking**

19. **EKS Service Troubleshooting**
20. **CoreDNS / DNS**
21. **NetworkPolicy**
22. **ALB 404**
23. **ALB 502/503**
24. **LoadBalancer Pending**
25. **VPC CNI**
26. **PVC / StorageClass / EBS CSI**
27. **EBS Attach / Multi-Attach**
28. **StatefulSet Storage**
29. **Disk & Volume Permissions**
30. **CPU Throttling**
31. **Requests & Limits**
32. **HPA + Metrics Server**
33. **Cluster Autoscaler**
34. **ConfigMap / Secret / Environment**
35. **YAML / Schema / API Errors**
36. **EKS RBAC and Access Entries**
37. **ServiceAccount: RBAC vs AWS IAM**
38. **TLS / Certificates**
39. **Node NotReady / kubelet**
40. **Node Pressure**
41. **EKS API Connectivity**
42. **Why etcd/kubeadm Troubleshooting Doesn't Apply to EKS**
43. **Managed EKS Upgrade Troubleshooting**

The biggest improvement is the reasoning chain: students should be able to explain **“I ran this command because I am testing this hypothesis; this output proves this component failed; this change fixes the specific constraint; and these checks prove recovery.”** That is the level to keep across the course rather than teaching command memorization.

Current AWS EKS References Used in the Material

- Amazon EKS IAM for add-ons and workloads: <https://docs.aws.amazon.com/eks/latest/userguide/add-ons-iam.html>
- Amazon VPC CNI: <https://docs.aws.amazon.com/eks/latest/userguide/managing-vpc-cni.html>
- EBS CSI on EKS: <https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html>
- EKS Access Entries: <https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html>
- EKS control-plane logs: <https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html>
- EKS managed node group updates: <https://docs.aws.amazon.com/eks/latest/userguide/update-managed-node-group.html>
- EKS managed node update behavior: <https://docs.aws.amazon.com/eks/latest/userguide/managed-node-update-behavior.html>